ON METACRAFT

The Reimplementation Cost Collapse

Sandra Schipal

draft

On MetaCraft
Sandra Schipal

Draft. Not for sale.

Typeset with DesignCraft. Print profile: mcp-central-docs/standards/BOOK_PRINT_PROFILE.md.

Contents

Preface · 5

I. The Breach · 7

II. The Mechanizers · 11

III. No Moat Equation · 15

IV. The Stack · 19

V. Locating the Present (living, revision-dated) · 25

VI. Thresholds · 33

VII. Dark Megawatts · 39

VIII. The Physical Ladder · 43

IX. MetaCraft · 49

X. The Physical Firewall · 55

Side: The Dark Mirror (governance only) · 57

Side: The Reversing Foundry – Inverse Engineering and the Leapfrog Craft · 61

Side: October 2026, Red-Letter Month - Reversing and Games Fall as Crafts · 67

Side: The AAA Collapse, the Bangalore Axiom, and the Curation Oracle · 73

Side: Two Hours of Digibib5 · 79

Side - Glossary · 81

Side - The Gemini First Draft (frozen 2026-10-13) · 83

ON-METACRAFT · 83

Review A - Gemini first draft (Opus 5.5) · 91

Review B - ArtCraft Crafting Apps, verified (Opus 5.5) · 95

Review C - The patent objection (Opus 5.5) · 99

Review D - Checking cost, lawyers, defection (Opus 5.5) · 103

Review E - The physical ladder and MetaCraft (Opus 5.5) · 107

Review F - Cross-Model Assessment and The Four Questions (Gemini) · 113

Preface

This book was started on 2026-10-13, thirteen days after the first commit to a repository called `storytold/photocraft`. By the time the first draft existed, the same organization had published thirteen desktop applications, each aimed at a named incumbent, each released roughly once a day. The book is therefore written from inside the event it describes. Some of its dates will be wrong by the time anyone reads it. Chapter V exists to absorb that: it is a living ledger, revised in place and dated at every revision.

Two levels of authorship

The first draft of the argument came from Gemini, in a chat chain, on 2026-10-13. It is preserved word for word in Side - The Gemini First Draft, in this book. Claude Opus 5.5 reviewed it the same day, checked its claims against public repositories and press, and disagreed with parts of it. Those reviews are kept as dated side chapters (Sides - Reviews A-F), never edited after their date. Sandra Schipal judges between the models and owns the text.

This arrangement is not decoration. A book about software built by agent swarms, drafted by competing models and judged by one human, is a small instance of its own subject. Where a model's judgment survives into the text, the commit carries a `Co-Authored-By:` trailer.

What this book claims

Proprietary desktop software was defended less by law, and less by genius, than by the labor cost of reimplementing it. In 2026, agent harnesses running on cheap compute collapsed that cost. The law had permitted clean-room reimplementation for decades. What remains of the moat after the collapse is switching cost, distribution, certification, liability, and a narrow, decaying band of patents.

What this book does not do

- It does not give legal advice. Where it discusses doctrine, it cites the case and the date. Ask counsel.

- It does not give investment advice. Chapter VI separates exposed vendors from defended ones; that is analysis, not a trade.
- It does not teach offensive technique. The same harness that clones a photo editor can be pointed at contested domains. The book names those domains once, in Side - The Dark Mirror, at the level of institutions and governance only (Rule 2, `CONTRIBUTING.md`).

How to read it

Chapters I-V establish what happened and when. Chapter VI lists what still holds and who holds it. Chapter VII is about the compute that pays for the collapse. Chapters VIII and IX follow the method off the screen and up the autonomy ladder. Chapter X sets the physical limits.

Every unverified claim carries a `[construction]` flag. A chapter with no flags left is a finished chapter. At 2026-10-13 there are none.

Two falls

I witnessed the fall of communism in Prague. I walked up to Hradcany, to Hradcanske namesti in front of Prague Castle and St. Vitus Cathedral, on a winter night when a million people in the city below chanted freedom, freedom. The square before the presidential palace was completely empty. I thought great thoughts. I was elated. A column of military trucks came up, armored carriers appeared, drove the length of the square, turned, rolled past me. The soldiers looked at me, I looked at them. Nobody shot me. One of the great moments of my life.

I am witnessing the second great fall now. Then it was a monopoly on power that looked permanent until it was not. Now it is a monopoly on expertise that looked permanent until it is not. This book is written from inside that second fall.

Fleet link

The sister books (`on-ai-takeover`, `on-hyperwar`, `on-hyperart`) use the same ladder method on the economy, on war and on art. This one is about the tools.

I. The Breach

`storytold` org, PhotoCraft README / AGENTS.md / ROADMAP.md, press 2026-10-07..18, Goliath install check 2026-10-18, `mcp-central-docs/projects/adobe-offensive/01` (2026-10-08).

Thirteen days

On 2026-09-30 a developer named Brandon Thomas made the first commit to a repository called `storytold/photocraft`: an image editor, written in Rust from scratch, described as a clean-room reimplementation of Adobe Photoshop. Four more repositories were created the same day: VectorCraft for Illustrator, FilmCraft for Premiere, LightCraft for Lightroom, PdfCraft for Acrobat. EffectCraft and DesignCraft, for After Effects and InDesign, followed the next day. All seven shipped their first releases on 2026-10-02. The suite went public at getartcraft.com/apps over the weekend of 2026-10-04 `[construction: confirm site launch date]`.

On 2026-10-07 Ars Technica ran the story under the headline “Software is over”. The same day, five more repositories appeared: WordCraft, GridCraft and DeckCraft for Microsoft Word, Excel and PowerPoint; SoundCraft for Avid Pro Tools; CADCraft for Autodesk AutoCAD. FigCraft, for Figma, followed on 2026-10-09.

By 2026-10-12 the thirteen apps had published 93 releases between them, about one release per app per day. PhotoCraft had 55,197 GitHub stars in the 2026-10-13 snapshot. The 2026-10-18 snapshot (`evidence/2026-10-18.json`): 67,027 stars, 12 releases, latest tag v0.10.0; suite-wide 120 releases and 167,586 stars. Inflow cooled after an Oct 7-12 spike. Stars are weather. Chapter V holds the living count. PhotoCraft is dual-licensed MIT or Apache-2.0. The generative studio is a separate product, `storytold/artcraft`, not one of the thirteen.

Its commit log since 2026-09-30 held 1,511 commits and about 1,579 `Co-Authored-By: Claude` lines as of 2026-10-13: nearly every change had an agent as co-author. `[construction: re-count Co-Authored-By at next evidence run]`

Not a lone genius, not a corporation

The Gemini draft described the origin as a single experienced developer. That is half the picture. Thomas leads; press reports describe fifteen years of programming and about three years as chief executive of the AI start-up Storyteller.ai [construction: verify; funding history open]. He has said he worked exclusively with AI coding tools from early 2026 and settled on Claude Opus 5.5 [construction: primary source]. The commit log shows the rest: parallel agent instances writing most of the code, and, within days of launch, a crowd of human contributors. 227 distinct names had committed to PhotoCraft by 2026-10-11.

The correct unit is neither a person nor a company. It is a lead, a method and a swarm.

What was claimed, what was shipped

The founder's claims ran ahead of the software. Early posts spoke of reaching full parity with Adobe within a month; he later revised this to months rather than years [construction: HN thread ids in adobe-offensive/01]. Reviewers found the apps glitchy, incomplete and early. The project itself agrees: PhotoCraft's own roadmap, reviewed 2026-10-11, rates it about 45 percent ready for real work, with feature breadth near 76 percent.

Counting bugs on day thirteen measures the wrong thing. Discovery is being done for free by thousands of users filing issues; fixing is being done in parallel by agents; every fix lands in a shared Rust core used by all thirteen apps. The number to watch is not the bug count but the slope of the parity curve.

The number that matters

The PhotoCraft roadmap contains one line that should open every history of this period. It estimates the remaining distance to beta at 1,100 to 1,800 hours, measured in Opus 5.5 agent wall-clock hours for one agent working sequentially, calibrated from the repository's own 1,004 merged pull requests, and notes that about 80 percent of the work parallelizes.

Adobe's Photoshop organization measures its work in engineers and years [construction: dated headcount source]. PhotoCraft measures its remaining work in model time. That difference is the breach.

What the moat was

The Gemini draft opened with a sentence worth keeping: the moats were never laws of physics. They were the cost of human keystrokes. Chapter III shows the law never protected what the clones copy. Chapter IV shows how the method works. This chapter only fixes the date: between 2026-09-30 and 2026-10-12, the labor cost of reimplementing professional desktop software stopped being the barrier it had been for forty years.

Fleet link

on-ai-takeover: the per-seat subscription line item, seen from the buyer's budget, that this chapter makes optional. on-hyperart: the Crafts are how a generated picture becomes a PSD, AI, or PREL a client can still open. on-hyperwar: dual-use consumer stack versus exquisite incumbent; analogical only, no munitions software.

II. The Mechanizers

revised per Reviews A and C. Gemini and Sandra review pending.

The guild's defense

Every monopoly on making things has defended itself the same way when a machine arrived: the machine's output is inferior. The claim is usually true on the first day and false within a decade. The interesting question is never whether the claim is true but how long it stays true, and what the incumbent does with the time.

The pattern has six dated instances worth knowing.

Lyon, 1804

Joseph Marie Jacquard's loom controlled the lifting of warp threads with a chain of punched cards `[construction: patent and demonstration date, 1801-1804]`. Before it, a figured silk required a drawboy perched on the loom, pulling cords in sequence for hours. The pattern lived in the drawboy's attention. After it, the pattern lived in a stack of cards that could be copied, stored and run again.

The weavers of Lyon resisted; looms were reportedly burned. The resistance failed for a reason that recurs in this book: the cards were cheaper to check than the drawboy. A card either has a hole or it does not. The pattern became data, and data can be verified.

Highland Park, 1913

Ford's moving assembly line did not invent the automobile. It removed the coachbuilder's monopoly on assembling one. Interchangeable parts, made to tolerances a gauge could check, let unskilled workers do in minutes what had taken a craftsman hours `[construction: Model T chassis assembly time before/after, 1913-1914]`. Again the enabling condition was cheap verification: a part fits the gauge or it does not.

Cambridge, Massachusetts, 1983

On 1983-09-27 Richard Stallman announced the GNU project, a free reim-

plementation of Unix. On 1991-08-25 Linus Torvalds announced a kernel to go with it. Together they broke the licensing monopoly on Unix, not by lawsuit but by rewriting it, interface by interface, against a published standard (POSIX) and an observable original.

This is the closest historical precedent to ArtCraft, and it is also a warning. GNU/Linux won the server and lost the desktop. It reimplemented the interface programs used, not the interface people used. Chapter III returns to that difference; it is the hinge of the book.

Toronto and San Diego, 1986

Rumelhart, Hinton and Williams published backpropagation in *Nature* on 1986-10-09. It took until 2012, and a GPU, for the method to beat hand-built feature engineering decisively (AlexNet, ImageNet 2012). The monopoly broken was that of the expert who wrote the rules by hand. The mechanism was statistical learning from data the expert did not curate.

Santa Clara, 2006

NVIDIA's CUDA `[construction: release date, 2006-11 or 2007-02]` turned a graphics card into a general matrix engine. It broke the dominance of scalar general-purpose processors for numerical work. Without it, neither the 2012 result nor the 2026 agent swarms exist.

San Francisco, 2026

The sixth mechanizer has no single name. It is a method: a parity document scored against the installed original, a reference implementation the agents can test against, a budget in agent-hours, and a swarm of model instances writing code in parallel. Chapter I dates its public arrival to 2026-09-30.

The trade it mechanizes is programming itself. Between 2023 and 2025, AI coding was an industry joke: autocomplete snippets that hallucinated libraries, broke on multi-file refactors, and required constant human correction. In 2026, agentic harnesses with closed compiler feedback loops crossed into superhuman speed and consistency. The craft of software construction became the first solved trade.

The monopoly it breaks is the labor cost of reimplementation. The guild de-

fense has already been mounted: the clones are buggy, incomplete, alpha. All three are true on day thirteen.

A counter-case: Bessemer

Not every mechanizer was robbed. Henry Bessemer patented his converter process in 1855-1856, licensed it, and is estimated to have earned over a million pounds in royalties before the patent expired in 1870. When William Kelly's rival American patent appeared, it was bought and pooled with Bessemer's; mills paid under both. The law worked as designed.

Bessemer matters here because he is the exception that shows the rule's real shape: patents protected a physical process with a twenty-year clock. The clones of Chapter I attack software whose core is older than twenty years. Chapter III takes up the difference.

The editorial correction

The Gemini draft listed Dario Amodei as a mechanizer, for constitutional scaffolding of frontier models. That is a brake, not a mechanizer: it constrains a machine; it breaks no guild. It moves to Chapter VI as an attachment point.

Fleet link

on-hyperart Ch II traces the same "inferior imitation" defense in painting, voice and animation. Same defense, same clock.

III. No Moat Equation

(rebuilt), Review C, `mcp-central-docs/projects/adobe-offensive/14-legal-deep-dive.md` (2026-10-11), Sandra on adhesive ToS 2026-10-18. Not legal advice.

The wrong question

The Gemini draft called the incumbents' legal position a bluff: you cannot copyright a Bezier curve. That is true and beside the point. Copyright never protected the Bezier curve, the alpha-compositing formula or the matrix multiply. Nobody believed it did. The law was not the moat.

The right question is why, if reimplementation was legal for forty years, it was so rare.

What the law always allowed

The doctrine is old and consistent.

- **Ideas are free.** 17 U.S.C. s.102(b) excludes any idea, procedure, process, system or method of operation from copyright. *Baker v. Selden*

  (1879) drew the line between a bookkeeping system and the book that explained it.

- **Menus are a method of operation.** *Lotus v. Borland* (1st Cir. 1995, affirmed by an equally divided Supreme Court in 1996) held the Lotus 1-2-3 menu command hierarchy uncopyrightable.
- **Studying the original is allowed.** *Sega v. Accolade* (9th Cir. 1992) and *Sony v. Connectix* (9th Cir. 2000) held intermediate copying for the purpose of understanding unprotected elements to be fair use. Both defendants disassembled code. Watching a program run is milder.
- **Reimplementing an interface can be fair use.** *Google v. Oracle* (US Supreme Court, 2021-04-05) held Google's reuse of Java's declaring code fair use, weighing the investment programmers had made in learning the interface.

- **In Europe, observation is a right the license cannot remove.** Software Directive 2009/24/EC Art. 5(3) lets a licensee observe, study and test a program to determine its underlying ideas; Art. 8 voids contract terms to the contrary. *SAS Institute v. World Programming* (CJEU C-406/10, 2012-05-02): functionality, programming language and data file formats are not protected; a licensee who observed a learning edition to reproduce functionality acted lawfully.

The Phoenix BIOS clean room (1984) is the industrial template: one team studies the original and writes a specification; a lawyer vets it; a second team that never saw the original writes the code. The wall exists to rebut an inference of copying. It is evidence, not a legal requirement.

So why was it rare?

Because it was expensive. A clean-room Photoshop was always lawful and always a decade of skilled labor. GIMP (1996), Krita, Inkscape and LibreOffice show what volunteer labor could reach: capable programs that deliberately did not imitate the incumbents' interfaces. They reimplemented functions. They did not reimplement habits.

That is the moat this book is about. **Switching cost** is the time a trained user loses moving to a program that works differently. For a professional with ten thousand hours in Photoshop, it is the single largest cost of leaving. Microsoft Office users did not stay away from LibreOffice because LibreOffice could not open their files. They stayed because their hands knew where things were.

Google v. Oracle is the doctrine for exactly this cost: the Court weighed the investment users had made in learning an interface as a reason reimplementation was fair. PhotoCraft's own pitch is the same argument told for end users: the menus, shortcuts and panels your hands already know. The legal theory and the product strategy are one sentence.

What the clones change

The ArtCraft method (Chapter IV) attacks switching cost directly. Its parity documents score each app against the installed incumbent, menu item by menu item. At 2026-10-11 PhotoCraft reported every Photoshop menu item present (628 of 628) and 56 of 68 tools. A faithful clone, accurate on the sur-

face and several levels down, makes the trained hand portable.

Agents made that affordable. Nothing in the law changed.

What still binds

Four legal surfaces survive the clean room. They are thresholds, handed to Chapter VI with owners and dates.

1. **Patents.** Independent creation is no defense. The surface is real but narrow and decaying: twenty-year terms, *Alice v. CLS Bank* (2014-06-19) against abstract software claims in the US, and the EPO's technical-character test, which since *T 208/84 Vicom* (1986-07-15) has treated image processing as technical. Live risk clusters in features newer than about 2006, and those are the features the clones lack.
2. **Trade dress.** Lanham Act s.43(a). The most likely first move by an incumbent is a letter about look, not code. The defense is that familiarity is functional; the cheap remedy is to restyle the chrome.
3. **Contract.** End-user license clauses against reverse engineering can bind a US licensee who clicked (*Bowers v. Baystate*, Fed. Cir. 2003). In the EU, Art. 8 voids them as far as observation goes. They are not a moat against clean-room reimplementation. See below.
4. **Format licenses.** Mostly waived by the incumbents themselves to win ISO standardization: Adobe's royalty-free patent license for ISO 32000-1:2008 (PDF 1.7), Microsoft's Open Specification Promise for OOXML (scope disputed by FSFE and SFLC in 2008). PDF 2.0 (ISO 32000-2,

   2017) and Adobe XFA sit outside the PDF license.

Where the AI clean room is weaker than the label

Honesty requires two caveats.

- **The wall.** If one model context both watched the original and wrote the code, there is no wall in the Phoenix sense. Separate observer and implementer contexts with a vetted specification between them would rebuild it. Whether ArtCraft does this is not public. `[construction]`
- **The weights.** The model was trained on public material that includes

tutorials, documentation and screenshots of the incumbent. Clean cannot be proven at the level of weights. The test remains the output: protected expression must be shown in the clone. Fresh Rust against decades-old C++ is unlikely to show it.

As of 2026-10-18 no cease-and-desist, takedown or suit against the Crafting Apps had been reported in press roundups `[construction: re-check github/dmca and dockets at each revision]`.

The ToS that is not a threshold

Adobe's terms, as discussed in October 2026 coverage, include language forbidding monitoring or tracking an application's inputs and outputs "in order to recreate that system." Counsel pieces treat this as a live risk. This book does not.

A term that says "do not look at our product in order to compete" is not a property right. It is a contract of adhesion trying to claw back what copyright already excluded: ideas, procedures, methods of operation (s.102(b)), observation of a running program (Sega, Connectix; Directive Art. 5(3)), and, in Europe, a right the license cannot remove (Art. 8, SAS). Sandra's position, dated 2026-10-18: the clause is arbitrary, based on thin air, a no-hoper as a moat for Adobe or any other vendor. A letter may still be sent. Distribution (Chapter VI, threshold D) may still pinch. The term itself does not convert looking into infringement.

Bowers at most binds the person who clicked that EULA. It does not bind a team that never took the license. It does not survive Art. 8. This chapter therefore does not promote anti-clone ToS language to an attachment point.

Fleet link

on-hyperart Ch VII: copyright filters as a failed control case. There the law tried to protect style and could not; here it never tried. The ToS move is the same wish in contract clothes.

IV. The Stack

PhotoCraft README / AGENTS.md / ROADMAP.md (2026-10-11), Review B. Gemini and Sandra review pending.

The inversion

A desktop application used to be a window with a program behind it. The window was the product; the program existed to serve the mouse. The Crafting Apps invert this. The program is a document model and a list of commands. The window is one of four ways to call those commands, and not the most important one.

```
human intent / agent
|
v
command registry  <---  UI (egui)  |  CLI  |  JSON channel  |  MCP
server
|
v
document model  ->  CPU reference compositor  <== compare ==>  GPU
compositor
|
v
files (PSD, PDF, DOCX, ...)  /  pixels  /  audio  /  drawings
```

Everything is a command

PhotoCraft's contributor rules start with one sentence: everything is a command. Every user-visible behavior is registered once in the engine, with an id, a label, a menu path, a shortcut, documented parameters, an enabled-check and a run function. The UI, the command line, the JSON control channel and the MCP server all dispatch the same ids. At 2026-10-11 the registry held more than 500 commands.

The consequence is easy to miss. Agent control is not a feature added after the fact. An app built this way cannot have a function the agent cannot reach, because there is no other way to add a function. The Gemini draft called MCP "the real nervous system". It is more exact to say there is only one nervous system, and the human hand is one of its four inputs.

Goliath, 2026-10-18: Sandra reports CLI and MCP on the installed Crafts are usable. That is an operator fact about this machine, not a benchmark. Cursor's user `mcp.json` on Goliath does not register the Craft servers; the control plane is the app's own CLI / JSON / MCP, as the READMEs describe.

Native, not wrapped

The apps are written in Rust only. The rules forbid JavaScript, webview shells and Electron. The desktop build draws with egui on wgpu; the web build is the same Rust compiled to WebAssembly. One codebase reaches macOS, Windows, Linux and the browser; PhotoCraft also builds for FreeBSD. FilmCraft alone had no web build at 2026-10-13.

The Gemini draft promised cold starts under twenty milliseconds and memory in megabytes rather than gigabytes. That may be true; it is not yet measured in public. `[construction: measure named app vs incumbent on named hardware; PhotoCraft has a perf/ directory]`

The oracle

This is the part the Gemini draft missed, and it is the part that makes the rest possible.

PhotoCraft composites images twice. A CPU compositor, written for clarity, is the reference. A GPU compositor, written for speed, is tested against it. The PSD reader is tested against files authored in Photoshop itself and kept in a separate, checksum-pinned corpus. Every command is fuzzed with adversarial parameters by a test called `panic_hunt`; a crash on bad input fails the build. A parity floor records how many incumbent menu items are live and may only rise.

An oracle is anything that can say, cheaply and without a human, whether the output is right. Agents can write code quickly. What they cannot do without an oracle is know when they are wrong. Every part of this stack that an agent touches has one.

Rule, stated once here and used for the rest of the book: **agent synthesis scales exactly as far as cheap verification reaches.**

The black-box oracle: screen-only extraction

The oracle described above assumes the incumbent sits on the same disk: an installed binary whose file formats and memory buffers can be inspected. What happens when the incumbent cannot be inspected at all – when the target runs on locked console silicon in Yokohama or inside an encrypted cloud data center?

The answer is the screen-only oracle.

An application or game running in a zero-trust enclave still has to emit photons to a screen and sound waves to an audio device in response to user inputs. An autonomous agent can drive that interface through standard input queues, recording video at 120 frames per second. After a billion input-frame pairs, the black box yields its secrets without a single byte of machine code being decompiled:

1. **Kinematics via symbolic regression.** By stepping a character through jumps, collisions, and slopes, pixel velocities are tracked frame-by-frame. Symbolic regression fits the exact gravity constants, acceleration parabolas, and collision boundaries.
2. **State machines via automated traversal.** Every menu branch, dialog tree, hit confirm, and inventory transition is mapped as a directed state graph:

3. **Geometry via inverse rendering.** A camera traversing a virtual space captures dense multi-view footage. Neural radiance fields (NeRFs) and 3D Gaussian Splatting reconstruct collision meshes, polygon geometry, and PBR textures directly from the video stream.
4. **Audio demixing.** Real-time neural separation isolates sound effects, dialogue, and score stems.

The output can be synthesized into a deterministic Rust/Vulkan engine, or loaded into a real-time generative world model (the path demonstrated by Google's GameNGen and Decart's Oasis in late 2024).

This is the cleanest clean room in software history. It touches no memory, bypasses no encryption, and triggers no anti-circumvention provisions under 17 U.S.C. s.1201. The incumbent is not disassembled; it is simply observed by an entity that does not blink.

Reversing as the daughter of coding

The reversing revolution is not an independent technical event. It is the next rung of the coding revolution.

When an agent can synthesize complex, multi-threaded systems software in seconds, reverse engineering ceases to be a manual crawl through hex dumps. The agent approaches binary analysis as an inverse compilation problem: it inspects header signatures, writes an ad-hoc Rust unpacker in twenty seconds, executes it against the binary, observes where the decompression fails, fixes the offset, and runs it again.

The 100-gigabyte game archive or encrypted firmware blob that defeated human teams through sheer volume is trivialized by this loop. An agent does not drown in data because it generates the machinery to process the data on the fly. Forward code synthesis makes inverse code synthesis cheap. (Side - The Reversing Foundry develops the complete two-tier clean room architecture; Side - Two Hours of Digibib5 records a concrete instance on this machine: twenty years of German CD-ROM container obscurity dissolved by Claude Opus in 120 minutes.)

The rules agents follow

The code rules read like a contract written for machines:

- No `unsafe` code anywhere but one isolated crate for pen-tablet input.
- No `unwrap`, `expect` or `panic` outside tests, enforced by the linter.
- No assumption of 8-bit color or sRGB in any public interface.
- No proprietary code, shaders, profiles or assets; implement from public specifications and observation.
- Every change comes with tests; every crash fix with a test that crashed before the fix.

Shared rules across all the Crafting Apps live in a private repository called `craftrules`. Shared Rust libraries live in `craft-libs`, pinned by revision.

The budget

The PhotoCraft roadmap does something no software plan known to this book has done before. It gives the distance to beta, 1,100 to 1,800 hours, in Opus

5.5 agent wall-clock hours for one agent working sequentially, calibrated from 1,004 merged pull requests, and notes that about 80 percent of the work parallelizes.

A project that budgets its future in model time has changed what a software project is. Labor has become a line item priced in tokens. Chapter VII is about who pays for them.

Fleet link

on-ai-takeover: MCP as the interface that turns seat licenses into API calls. The buyer's side of the same stack.

V. Locating the Present (living, revision-dated)

Rule: one row per dated public artifact. No row without evidence. Public catalog: getartcraft.com/apps (“Thirteen apps. One craft.”), accessed 2026-10-18. GitHub org `storytold`. Machine check: hostname Goliath, ArtCraft Launcher v0.5.1.

What the thirteen Crafts are

App	Incumbent workflow	What it is (project’s words, 2026-10-18)	Repo created
PhotoCraft	Photoshop	Image editing: layers, masks, type, real PSD files	2026-09-30
VectorCraft	Illustrator	Vector illustration	2026-09-30
FilmCraft	Premiere	Video editing, color and sound	2026-09-30
LightCraft	Lightroom	Photo library and raw development	2026-09-30
PdfCraft	Acrobat	Reading, organizing and protecting PDFs	2026-09-30
EffectCraft	After Effects	Motion graphics and visual effects	2026-10-01
DesignCraft	InDesign	Page layout and publishing	2026-10-01

WordCraft	Word	Word processing; ribbon, styles, .docx	2026-10-07
GridCraft	Excel	Spreadsheets and data	2026-10-07
DeckCraft	PowerPoint	Presentations and slide decks	2026-10-07
SoundCraft	Pro Tools	Audio recording and editing	2026-10-07
CADCraft	AutoCAD	CAD/drafting; command line, snaps, DXF/DWG	2026-10-07
FigCraft	Figma	Interface design and prototyping	2026-10-09

Shared conventions, stated by the project: clean-room, pure Rust, native on macOS/Windows/Linux, WebAssembly except FilmCraft as of 2026-10-13, agent-drivable (CLI, JSON channel, MCP). Dual MIT/Apache-2.0 on PhotoCraft; CADCraft and WordCraft READMEs say the same. Early alpha. PhotoCraft's own 2026-10-18 README: not yet a Photoshop replacement for daily professional work.

Not in the thirteen, and not a Crafting App:

- **ArtCraft** (`storytold/artcraft`, created 2022-03-31): generative image/video studio ("IDE for artists"). Credits and subscriptions live here, not on the clones. Evidence filter in `book.toml` excludes this repo on purpose.
- **ArtCraft Launcher** (`storytold/craft-launcher`): installer/updater. Goliath: v0.5.1, published 2026-10-18T04:01:09Z, binary at `%LOCALAPPDATA%\Programs\ArtCraft Launcher\artcraft-launcher.exe`.

Hand-kept first-release table (2026-10-13, unchanged)

Repo created	App	Incumbent	First release	Evidence
2026-09-30	PhotoCraft	Photoshop	2026-10-02	github.com/storytold/photocraft
2026-09-30	VectorCraft	Illustrator	2026-10-02	github.com/storytold/vectorcraft
2026-09-30	FilmCraft	Premiere	2026-10-02	github.com/storytold/filmcraft
2026-09-30	LightCraft	Lightroom	2026-10-02	github.com/storytold/lightcraft
2026-09-30	PdfCraft	Acrobat	2026-10-02	github.com/storytold/pdfcraft
2026-10-01	EffectCraft	After Effects	2026-10-02	github.com/storytold/effectcraft
2026-10-01	DesignCraft	InDesign	2026-10-02	github.com/storytold/designcraft
2026-10-07	WordCraft	Word	2026-10-07	github.com/storytold/wordcraft
2026-10-07	GridCraft	Excel	2026-10-08	github.com/storytold/gridcraft
2026-10-07	DeckCraft	PowerPoint	2026-10-08	github.com/storytold/deckcraft
2026-10-07	SoundCraft	Pro Tools	2026-10-08	github.com/storytold/soundcraft

2026-10-07	CADCraft	AutoCAD	2026-10-08	github.com/storytold/cadcraft
2026-10-09	FigCraft	Figma	2026-10-10	github.com/storytold/figcraft

Goliath install (dated 2026-10-18)

Hostname `Goliath`. ArtCraft Launcher v0.5.1 installed. Roaming app-data folders present under `%APPDATA%` for PhotoCraft, VectorCraft, FilmCraft, LightCraft, PdfCraft, DesignCraft, WordCraft, GridCraft, DeckCraft, SoundCraft, CADCraft, FigCraft. PhotoCraft log: version 0.9.0 launched 2026-10-15T23:08:56Z. Sandra states CLI and MCP on the installed Crafts are usable on this machine as of 2026-10-18. Cursor's user `mcp.json` on Goliath does not list Craft servers; agent control is per-app (CLI / JSON / MCP), not a Cursor-global registration.

A `%APPDATA%\lookcraft` folder exists (first seen 2026-10-15). No public `storytold/lookcraft` repository found 2026-10-18. Not added to the thirteen. `[construction: identify or ignore]`

EffectCraft has no roaming folder on Goliath at this check. Website still lists it. `[construction: confirm install vs unused]`

Live snapshot (generated)

Regenerated by `just evidence` from the GitHub API. The tables above are the hand-kept record; this one moves with each snapshot.

Snapshot 2026-10-18 (`evidence/2026-10-18.json`).

Repo created	Repo	Target	First release	Releases	Latest	Commits	Stars
2026-09-30	storytold/filmcraft	Premiere	2026-10-02	10	v0.9.0	2015	14722

2026-09-30	story-told/lightcraft	Lightroom	2026-10-02	10	v0.9.1	2913	13590
2026-09-30	story-told/pdfcraft	Acrobat	2026-10-02	10	v0.9.0	1237	15710
2026-09-30	story-told/photocraft	Photoshop	2026-10-02	12	v0.10.0	2564	67027
2026-09-30	story-told/vectorcraft	Illustrator	2026-10-02	15	v0.13.1	4022	9777
2026-10-01	story-told/designcraft	InDesign	2026-10-02	10	v0.9.0	2595	4362
2026-10-01	story-told/effectcraft	After Effects	2026-10-02	12	v0.11.1	2060	7250

2026-10-07	story-told/cadcraft	AutoCAD	2026-10-08	7	v0.8.0	498	5696
2026-10-07	story-told/deckcraft	PowerPoint	2026-10-08	7	v0.8.0	909	3746
2026-10-07	story-told/gridcraft	Excel	2026-10-08	7	v0.8.0	1028	5682
2026-10-07	story-told/soundcraft	Pro Tools	2026-10-08	6	v0.8.0	692	3224
2026-10-07	story-told/wordcraft	Word	2026-10-07	7	v0.8.0	859	14239
2026-10-09	story-told/figcraft	Figma	2026-10-10	7	v0.7.0	1157	2561

Legal weather (silence, dated)

As of 2026-10-18 no public Adobe or Autodesk or Microsoft suit, GitHub

DMCA, or cease-and-desist against the Crafting Apps was found in press roundups (Ars 2026-10-07, Al Jazeera 2026-10-14, Declic). Counsel commentary clusters on trade dress, names, and patents, not on copyright of menus. See Chapter III on why an EULA "do not observe I/O to clone" clause is not treated as an attachment point.

Under construction: GameCraft (local, 2026-10-29) [construction]

Not public, not in storytold org, no HN post yet. Local repo D:, pure Rust workspace: gamecraft-core, gamecraft-commands, gamecraft-godot, gamecraft-mcp, gamecraft-cli, gamecraft-desktop (placeholder until Phase 4).

Status 2026-10-29: cargo check clean, cargo test 28 passed 0 failed (core 12, commands 4, godot 5, smoke 2, mcp 5), clippy clean, fmt clean. Sample fixture tests/fixtures/sample_3d_world.tscn carries intentional traps (RigidBody3D without shape, dangling signal target).

Rule for this entry: no storytold move, no HN note until Phase 5 depot gate holds - twelve playable Godot 4.3 exports under depot/games// with lint.json zero errors, trace.json win reachable, headful 60s clean. See PLAN-phase5-depot.md in gamecraft repo and side-red-letter-october-2026.md in this book.

Pending rows

- Reddit request for an AutoCAD clone: `[construction: thread URL + date; establish whether it preceded or followed 2026-10-07]` (Review D s.5).
- LookCraft roaming folder on Goliath, no public repo.
- EffectCraft install on Goliath.

Columns to add

Agent co-author share, parity % (from each repo's ROADMAP.md), incumbent price per seat per year at the same date.

VI. Thresholds

Sandra on ToS 2026-10-18. Not legal or investment advice.

Method

The sister books describe each surviving control as an attachment point: a place where a named party can still apply a brake, with a date for when it last held. This chapter lists the attachment points that survive the collapse of re-implementation cost. It is meant to be photocopied.

The table

#	Threshold	Who holds it	How it binds	State 2026-10-18
A	Patents	incumbents; USPTO, EPO, courts	method claims; independent creation is no defense	partial and decaying: 20-year terms; post-Alice; live claims cluster in post-2006 features the clones lack
B	Trade dress and trademark	incumbents; courts	look of the product; names and marks	real, cheapest first letter: restyle chrome, keep behavior. PhotoCraft's pitch is familiarity; that is the trade-dress surface
C	Format control	spec owners	undocumented or changing formats	weak where specs are public (PSD, OOXML, PDF 1.7); stronger for DWG history and project formats (AEP, PRPROJ)
D	Distribution	Apple, Microsoft, Google stores; code-signing authorities; GitHub	notarization, signing, store policy, takedown	strong: the realistic enforcement choke-point. Goliath currently runs the suite from the project's launcher, not a store

E	Certification and liability	regulators, insurers, auditors	software that must be certified or audited (medical, avionics, accounting, tax, stamped CAD)	strongest: the process, not the output, is audited. CADCraft is the Craft that hits this first
F	Enterprise bundle	CIOs; the incumbent's cloud	identity, mail, compliance, tenant data, support contracts, indemnity	strong: this is what enterprise software actually sells
G	Networks	banks, accountants, payroll and tax systems	the app is a node in a network the clone does not join	strong for QuickBooks-class targets
H	Model-provider policy	labs	usage policies; constitutions	variable; outside the clone's control

Anti-clone EULA language ("do not observe I/O to recreate") is **not a row**. Chapter III: it is a wish, not a property right. It does not get a letter in this table.

Row H is where the Gemini draft's Amodei line belongs: a brake on the mechanism, held by the lab, not a mechanizer.

Reading the table

CADCraft is the liability exhibit. An ugly PSD is an ugly PSD. A DWG that an agent stamped into a permit packet is a named professional's problem. Threshold E is not "certification" as a slogan. It is: who answers when the drawing is wrong.

Thresholds A to C are the old moats. They were never strong; labor cost hid how weak they were. The clones expose them. Contract terms that forbid looking are not in A-C. They are noise around D.

Thresholds D to G are not about the program. They are about what surrounds it: who may ship it, who must certify it, which network it joins, which bundle it sits in. They are where the incumbents will move, the way the music industry moved after 2001 from selling copies to licensing catalogs through

a service. The law stopped Napster. It did not stop file sharing. The industry survived by finding a moat that still worked.

Who is exposed

The useful split is not by size or sector but by where the moat sits.

- **Exposed:** vendors whose moat is the application itself: its interface, its features, its file format. Desktop creative tools, standalone office applications, single-user design and audio tools. Above all: **the AAA video game and console industry**. For decades, game publishers assumed a 100-gigabyte encrypted blob of proprietary chunks and obscure shader bytecodes was unassailable by a hobbyist team. To twenty humans in a Discord server, that volume was a fortress. To a frontier agent, it is an afternoon: the agent does not read the gigabytes into its prompt; it writes purpose-built, multi-threaded Rust decoders and entropy sniffers on the fly, compiles them, and slices through the archive like a heated knife through butter. The entire single-player client catalog is exposed.
- **Defended:** vendors whose moat is a network, a bundle, data, a certification or a liability. Enterprise suites with identity and compliance attached; accounting with bank feeds and tax filing; certified engineering tools.

Most large vendors are both. Their exposed products are the ones they sell to individuals and small firms; their defended products are the ones they sell to enterprises. Expect defection from the bottom first.

Where the last brake sits

Threshold D deserves one more sentence because it survives every later chapter. Whoever signs the binary is accountable for it. Swarms can write code; a code-signing certificate is issued to a named party. When Chapter IX asks where the human remains in an autonomous software foundry, the first answer is here.

Fleet link

on-hyperart Ch VI and on-hyperwar Ch VI use the same table form. The

thresholds differ; the method is shared.

VII. Dark Megawatts

revised per Review A. Gemini and Sandra review pending.

Who pays for the agent-hours

Chapter IV ended with a budget in agent-hours. Somebody pays for them. The Gemini draft's answer is the most interesting economic claim in its text: the datacenter buildout of the mid-2020s, whatever it does to its investors, will leave behind cheap compute the way the telecom crash left behind cheap fiber.

The claim is half right. This chapter keeps the right half and dates the wrong one.

The fiber precedent

Between 1996 and 2001, carriers laid far more fiber than the traffic of the day required. When the bubble burst, much of it went unlit `[construction: share of US long-haul fiber unlit ca. 2002, dated source]`. Bankrupt carriers sold their networks for a fraction of what they cost. The buyers lit the fiber cheaply over the following decade, and the web of video, cloud and mobile data ran on capacity that someone else had already written off.

The mechanism is general: capital that is destroyed on the balance sheet is not destroyed in the world. A written-off asset that still works has near-zero capital cost to its next owner.

The right half

A GPU written down to zero on an accountant's schedule still computes. Depreciation is an accounting convention; it is not a physical clock. When a newer generation arrives, older racks that are fully paid for do not vanish. Their owners can sell their time cheaply, and for work that does not need the lowest latency, such as batch code synthesis, test runs and fuzzing, they are good enough.

That is precisely the workload of Chapter IV. The agent swarms that build the Crafting Apps do not need to answer a user in two hundred milliseconds. They need many hours of patient compute, and they can wait for it overnight.

The wrong half

The Gemini draft claimed silicon does not rot. Two facts disagree.

1. **Failure at scale.** Meta's Llama 3 report (2024-07) recorded hundreds of unexpected interruptions during a 54-day training run on about 16,000 H100 GPUs, a majority attributed to GPU and memory faults `[construction: verify exact figures]`. Large GPU fleets fail continuously. Old fleets fail more.
2. **Electricity is not amortized.** A fully written-off accelerator still pays full price for every joule. When a newer chip produces several times more tokens per joule, the old rack loses on running cost alone, long before it breaks. Fiber was different: the glass got more valuable when better lasers were fitted at each end. A GPU cannot be upgraded at the ends. Its useful life is set by the efficiency of its successors.

So the dark-fiber analogy holds for years, not decades. `[construction: tokens per joule by generation, A100 / H100 / current, at a stated power price]`

What reaches the clone-builders

The subsidy only matters if it reaches the people running the swarms. That depends on who owns the stranded racks (hyperscalers, specialist cloud providers, lenders who repossess them) and whether spot prices for older hardware actually fall. The evidence to collect is a dated series of spot prices for previous-generation accelerators and of API token prices for the models used to write code `[construction]`.

There is a second subsidy the Gemini draft did not name: the model providers' own price competition. When frontier labs cut token prices to win developers, the cost of an agent-hour falls whether or not any datacenter goes dark.

The honest word

The Gemini draft's subtitle promised a zero-cost frontier. The PhotoCraft roadmap refutes it: 1,100 to 1,800 agent-hours to beta is not zero. The correct claim is that the **marginal cost of replication** is near zero and the **cost of synthesis** is falling fast. The chart this chapter needs is agent-hours to parity multiplied by the price of an hour, plotted over time. That curve, not the word zero, is the argument.

Fleet link

on-ai-takeover: the capital cycle from the investor's side. on-hyperwar: compute as a strategic stockpile.

VIII. The Physical Ladder

Software that makes things

Most professional software outside the office does not produce documents. It produces instructions for a machine: where a needle goes, which needle on a knitting bed lifts, where a blade cuts, how a tool head moves. The design program sits between a person and a machine, and it is often the most expensive and most locked part of the chain.

That makes it the next target. The method of Chapter IV does not care whether the output is a PSD file or a stitch file. It needs a spec, an original to compare against, and a cheap way to check the result.

The ordering rule

Chapter IV stated the rule: agent synthesis scales as far as cheap verification reaches. On a screen, verification is a pixel comparison. Off the screen, it is a physical test, and physical tests have prices. The ladder is ordered by them.

Rung	Domain	Physical check	Cost per check
0	Code itself (the solved baseline)	compiler, linter, test suite	fractions of a cent, milliseconds
1	Machine embroidery, home cutting	stitch-out or cut on scrap	cents, minutes, at a kitchen table
2	Machine knitting	a swatch	small
3	Electrochemical and spark machining (ECM/EDM)	bore or cut on scrap stock	dollars, minutes to hours

4	Garment pattern and fit	bodies vary, cloth drapes	medium; needs people
5	Kinematic assembly and tacit mechanics	3D-printed jig or test rig	tens of dollars, camera-in-the-loop
6	Agriculture	a growing season	a year
7	Chips	a tape-out	millions

Rung 0: code as the solved baseline

The ladder begins at zero because coding had to fall before any other craft could be attacked.

In software, the material and the tool exist in the same medium. A compiler either accepts a type or rejects it; a test suite either passes or fails. The cost of an oracle check in code is a CPU cycle. Because verification cost in software was effectively zero, programming was the first craft to collapse into superhuman agent throughput. Between 2023 and 2025, AI coding was an industry punchline; in 2026, it became the mechanizer for every trade above it.

Every physical rung that follows depends on Rung 0: software must be solved before software can drive physical machines.

Rung 1: needle and blade

Machine embroidery runs on stitch files (DST, PES, EXP and others) produced by digitizing software: Wilcom, Hatch, Brother PE-Design and others `[construction: current prices and licensing models]`. Home cutting machines run on design apps such as Cricut Design Space, which ties the software to the machine and to a subscription.

The formats are largely documented or long since reverse-engineered by open-source projects `[construction: name and date the main open libraries]`. The hard knowledge is physical: how much a stitch pulls the fabric, how density changes on stretch materials, how underlay holds a satin column. That knowledge is learned by stitching out samples. A sample costs a scrap of cloth and a few minutes. An agent with a camera and a machine can run that loop

all night.

Rung 1 is therefore not a distant future. It is plausibly the next step after office software.

Rung 2: the knitting bed

Industrial knitting machines (Shima Seiki, Stoll) are programmed through their makers' design systems `[construction: SDS-ONE APEX, M1plus, licensing]`. A program is a grid of needle actions. A swatch checks it. The loop is slower than embroidery and the machines are expensive, but the check is still small and local.

Rung 3: the bath and the arc

Subtractive metal machining usually demands heavy, stiff machine tools. Cutting forces deflect cutters, chatter ruins tolerances, and a rigid three-axis milling center costs tens of thousands of dollars. That capital requirement kept subtractive software protected behind commercial CAM licenses (Mastercam, Siemens NX, PowerMill).

Electrochemical machining (ECM) and electrical discharge machining (EDM) invert that requirement. Metal dissolves atom by atom into an electrolyte bath, or erodes across a dielectric spark gap. There is no contact force, no cutter deflection, and no chatter. A 3D-printed tool head carrying a copper or 3D-printed conductive cathode in a salt bath cuts hardened alloys that would destroy carbide end mills.

The barrier in ECM has always been the software: calculating cathode shape compensation for electric field gradients and electrolyte fluid flow. Historically, that math stayed inside specialized aerospace turbine programs. An agent harness running an open-source finite-element field solver can iterate cathode geometry against test coupons. The physical check is a saline wash over a scrap of hydraulic tubing, powered by a bench supply. The test costs two dollars in electricity and salt, takes forty minutes, and yields a clean micrometric cross-section.

Rung 4: the garment

Pattern-making software (Gerber AccuMark, Lectra Modaris, Optitex) and

3D garment simulation (CLO3D, Browzwear) turn bodies and cloth into flat pieces. Here the check gets expensive: fit depends on bodies, and bodies vary. The simulation is an oracle only as far as it is trusted, and fashion houses still sew samples. Rung 4 falls more slowly, and falls first where the body is standardized: uniforms, workwear, sizes from published tables.

Rung 5: tacit craft as compiler target

Human history is filled with lost crafts: techniques that vanished because their knowledge was tacit rather than codified. Historical metallurgical treatments, manual lathe-bed scraping to sub-micron flatness, precision watch escapement geometry, and the complex kinematic motions recorded in Renaissance combat treatises (such as the 15th-century Fechtbuecher) were transmitted by demonstration and muscle memory. When the guilds died, the craft died.

An agent harness approaches a lost craft differently. It treats historical texts, patents, and workshop notebooks as an informal specification. The agent compiles that specification into a parametric mechanical model in FreeCAD, exports the scene into an OpenUSD stage, and subjects the physics to a rigid-body simulator (MuJoCo or Isaac Lab).

The physical check on Rung 5 uses 3D-printed test fixtures, standard stepper actuators, and an inexpensive USB vision camera. The agent closes the loop between simulated kinematics and observed physical motion. Where a human apprentice required seven years under a master to acquire muscle memory, an agent harness with a camera evaluates fifty motion cycles an hour. The physical constraint is not human intuition; it is the wear rate of the actuator gears.

Rung 6: the field

The Gemini draft's SoilCraft (hydrology and soil models compiled to guidance tracks and irrigation schedules) meets a check measured in seasons. Worse for the clone, the moat in agriculture is not the planning software. It is the machine's firmware and the right to repair it. That fight is being fought in regulators and courts, not in code `[construction: FTC action against Deere, 2025, status]`.

Rung 7: the chip

Chip-design tools are the most expensive case, covered in Review D. The check is a foundry run that costs millions and the tools only count once a foundry certifies them. Lawyers do not protect this rung. Physics and the foundry do.

Rung 8: the biological test coupon (vibe genes and the assay wall)

Functional protein and genome design joined the collapse in late 2026 (Timmer, *Ars Technica*, 2026-10-28; *Pinal* and Hugging Face genomic models). Prompt-driven natural language compiles directly into candidate nucleotide and amino acid sequences: "vibe coding" applied to biology. The design software moat collapsed overnight.

What remains is the purest form of the physical firewall. An agent can generate ten thousand fluorescent protein candidates before lunch, but the check is a physical wet-lab synthesis and optical assay. Over 99 percent of prompted sequences fail to fold or express in vitro. The binding constraint is neither the model nor the software: it is the cycle time of liquid-handling robots, the cost of biological reagents, and the compliance screening imposed by DNA synthesis providers.

Where the moat goes

When the design software is free, lock-in moves to the machine maker: proprietary formats, cloud-required apps, encrypted firmware, warranty terms. Agriculture shows the pattern first. Expect embroidery and cutting machines next, and expect right-to-repair arguments to follow them from the farm to the sewing room.

The hand again

There is a cultural inversion in this chapter that the Gemini draft did not see. For thirty years, the expensive part of a craft hobby or a small textile business was often the software license. When the license is free, the scarce part is the human: the eye for cloth, the hand on the hoop, the judgment about drape. The tools return to the edge, and the skill becomes the thing worth having again.

IX. MetaCraft

The craft that makes crafts

Thirteen applications in two weeks is not thirteen projects. It is one method run thirteen times. This chapter names the method as an object of its own: MetaCraft, the foundry that makes Crafts under agent control.

It exists because coding was the first solved craft. Once the craft of writing software crossed into superhuman speed and consistency in 2026, software engineering ceased to be an artisan bottleneck and became an automated instrument of production. MetaCraft is what happens when that solved craft is directed systematically at every other trade.

The foundry already exists

Strip PhotoCraft's documents of their subject and what remains is a recipe:

Foundry part	ArtCraft instance (2026-10-11..13)	Physical / Greenfield instance (2026-10-24)
Specification	parity documents scored against the incumbent	STEP/URDF/OpenUSD scene graphs and constraint sets
Ground truth	the incumbent installed and measured (Photoshop 27.11.0)	physics engine oracles (MuJoCo, Isaac Lab) and test coupons
Budget	remaining work in agent wall-clock hours	GPU compute wall-clock for PPO training and FEA solvers
Checkers	reference implementations, command fuzzing	collision meshes, stress tensors, closed-loop vision feeds
Standards	shared rules across all apps (craftrules)	deterministic ROS 2 interfaces, headless CLI ergonomics
Output	about one release per app per day	containerized execution harnesses, calibrated G-code

A human chooses the target and approves the rules. Agents do most of the rest. The foundry is not a forecast. It is the commit log.

Five levels

The levels below are a way of dating progress. They are a proposal, not an industry standard.

Level	Who does what	Evidence at 2026-10-25
L0	Humans write the code	the old world
L1	Agents write code against specs and checkers made by humans	ArtCraft now
L2	Agents write the specification by examining the incumbent, and choose targets from demand	partial: the parity checklist is generated
L3	Agents build the checkers themselves	early evidence: automated synthetic physics test generation
L4	The foundry improves its own method, rules and pipeline	not yet evidenced in public

Foundry autonomy levels L0-L4: from humans write the code to the foundry improving its own method

Why L3 matters most

Chapters IV and VIII made one claim twice: agent synthesis scales as far as cheap verification reaches, and the order in which incumbents fall is set by the cost of checking a clone. Today, humans build the expensive checkers: the corpus of files made in the real application, the reference implementation, the physical sample.

A foundry whose agents build those checkers themselves removes the constraint that sets the order. Below L3, the storm moves through the industry at the pace humans can build oracles. Above it, at the pace of compute. Watching for the first public evidence of L3 is the single most useful thing a reader of this book can do. `[construction: define what counts as evidence, e.g. a corpus or reference implementation committed with no human author]`

The test bench paradox

A swarm that writes both the product and the tests can agree with its own mistakes. Independence needs a reference outside the swarm. In the cloning phase that reference is the incumbent: installed, measured, driven through the same actions as the clone, its outputs compared.

So the foundry depends on the thing it replaces. The incumbent becomes the clone's test bench. MetaCraft is strongest where an original exists and weakest where it must invent. What happens when the clone surpasses the original, and must become its own reference, is open. `[construction: Review E question 4]`

The greenfield physical stack

The foundry does not stop at two-dimensional pixel arrays or vector path canvases. In late October 2026, the method touched mechanical engineering and motion control.

The proprietary incumbents here are AutoCAD, SolidWorks, Revit, and CATIA. Their moats appeared wider than Adobe's because engineering software demands geometric kernel math (boundary representation, NURBS surfaces, constraint solvers) and tight integration with manufacturing hardware.

The greenfield response skips their graphical user interfaces entirely. The stack combines three headless components: 1. **Parametric geometry kernel**: Open-source engines (FreeCAD, OpenCASCADE, CadQuery) driven via Python interfaces. 2. **Universal scene description**: OpenUSD stages (`pxr` API) carrying kinematic hierarchies, material properties, and collision geometry. 3. **Synthetic physics oracle**: Massively parallel GPU simulators (Nvidia Isaac Lab, MuJoCo) executing reinforcement-learning test harnesses (RSL-RL PPO runners) alongside deterministic ROS 2 control nodes.

In this pipeline, verification is mathematical rather than visual. The agent does not check whether a rendered bracket looks pleasant. It applies a simulated ten-kilonewton shear load across a finite-element mesh and measures von Mises stress at the bolt holes. If the safety factor falls below 2.5, the harness mutates the fillet radius and reruns the solver.

Air-gapped execution and local sovereignty

Desktop proprietary software created a dependency structure through cloud entitlements, license servers, and mandatory telemetric telemetry. When an incumbent revokes a subscription, the user loses access to historical project files.

The greenfield foundry inverts that exposure. Because the agents gener-

ate pure code rather than binary blobs, the complete toolchain runs offline on consumer workstation hardware: an RTX 4090 desktop running WSL2 or Linux. No packets leave the local subnet.

This has an operational consequence that legacy vendors did not anticipate. When a engineering workflow requires zero external API calls and depends on no remote license managers, the toolchain becomes sovereign. An individual developer or small workshop operating an offline foundry can produce, verify, and output calibrated manufacturing toolpaths without external supervision or counterparty risk.

Where the humans remain

The sister books ask of every system where a named party can still apply a brake. In the foundry, the attachment points are:

1. **Target choice.** Who decides what to clone next.
2. **Rule approval.** Who signs off the foundry's standards.
3. **Release signing.** Who holds the code-signing certificate (Ch VI, threshold D).
4. **Accountability.** Who answers a court or a regulator.

As agents gain independence, these do not disappear. They concentrate into a few people and a few keys. A responsible history of MetaCraft names them and dates them, rather than describing the removal of the human from the loop. Removal is a claim about the code. Accountability is a fact about the law, and it has not changed.

The application does not die, yet

The Gemini draft predicted that the application would dissolve into ephemeral software, generated on demand and discarded. The evidence of 2026 runs the other way. The foundry produces stable, versioned clones of named products, released daily, tracked against a parity floor that only rises. That is more application, not less.

Disposable software may come later, for problems an automated check can verify completely. Where a process must be certified or audited (Ch VI, threshold E), the stable application survives because the audit needs something stable to audit. This book treats ephemeral software as a hypothesis for

a later revision of this chapter.

Fleet link

on-ai-takeover: the same autonomy levels applied to firms rather than applications. A company run as a foundry is the next chapter of that book.

X. The Physical Firewall

revised. Gemini and Sandra review pending.

What does not collapse

This book has described a collapse in the cost of making software. It closes by naming what the collapse does not touch, because a thesis that claims everything claims nothing.

Matter costs energy

Software copies at near-zero marginal cost. Steel, propellant, a five-axis milling machine and a chip fabrication line do not. Each needs mass, energy, capital and time that no prompt supplies. The Gemini draft put it well: you cannot prompt-engineer five tons of liquid oxygen.

The physical ladder of Chapter VIII shows the boundary in practice. The foundry advances quickly where the physical check is cheap (a stitch-out) and slowly where it is expensive (a growing season, a tape-out). The firewall is not a wall. It is a price, and the price rises with mass and energy.

The nanotechnology argument

The futurist tradition imagined software escaping into matter through molecular machines. The Gemini draft dismisses this, citing thermal noise, surface forces and heat. The argument has a dated public form: the 2003 exchange between K. Eric Drexler and Richard Smalley in *Chemical & Engineering News* `[construction: issue date]`. This book does not need to settle it. It needs only to note that in 2026 no software foundry produces matter, and that its effects on the physical world run through machines that already exist and are already owned.

Biology is slow

Designs made in software for living systems still need physical synthesis, reagents and growth, and they move at the speed of cells and of the institutions that oversee laboratories `[construction: cite one dated governance instrument, e.g. synthesis screening guidance]`. The collapse of software labor

cost does not collapse incubation time.

What did collapse

What collapsed is narrower and, for daily life, larger: artificial monopolies over information, coordination and tools. A trained user's habits, once locked to one vendor's program, became portable. A design program that cost more than the machine it drove became free. A software plan became a budget in model time.

The Gemini draft ended on three sentences. Two survive review. The moats are going, not gone; Chapter VI lists the ones that hold. The tools are returning to the edge: to the individual, the small firm, the crafter at a kitchen table. And the code belongs to whoever signs it, which is the attachment point every later revision of this book should watch.

The book in the corpus

This book was drafted by two models and judged by one person, in public repositories, within a fortnight of its subject. It will be read, if it is read at all, partly by models. Every claim is dated and every guess is flagged so that a later reader, human or not, can tell which is which. That is the only defense a book this close to its subject has.

Fleet link

on-hyperart Ch II: the futurists expected the physical singularity first; the informational one arrived instead. This chapter is that thesis applied to tools.

Side: The Dark Mirror (governance only)

Level 3. Governed by Rule 2 (`CONTRIBUTING.md`): institutional facts and governance criteria only. No mechanisms, now or in later revisions.

Why this file exists

The foundry is indifferent to its target. A method that clones a photo editor can be pointed at contested domains. A book that ignored this would be naive; a book that described how would be dangerous. This file does neither. It names the domains, the institutions that govern them, and the attachment points that apply.

Three contested domains

Domain	Main governing instruments	Attachment points
Offensive cyber capability	Wassenaar Arrangement controls on intrusion software (added 2013) [construction]; national computer-misuse law; model-provider usage policies	model providers, compute providers, code hosts
Weapons autonomy	US DoD Directive 3000.09 (updated 2023-01-25); UN CCW Group of Governmental Experts on lethal autonomous weapons; export controls	states, manufacturers, export authorities
Illicit commerce	anti-money-laundering and know-your-customer regimes (FATF standards); controlled-substance and precursor law	financial intermediaries, logistics carriers

Weapons autonomy is the subject of the sister book `on-hyperwar`, which owns that domain under its own hazard rule. This book defers to it.

The 2026 shift: automated reverse engineering and scaffolding CVEs

Six months after the spring 2026 Mythos security demonstrations, the labor cost of binary analysis and vulnerability discovery underwent the same collapse as desktop application development:

1. **The Reverse Engineering Tsunami:** Tools automating Ghidra and IDA Pro via MCP bridges (the fleet's `reversing-mcp`, Rea.ai) synchronized white-box decompilation with behavior clickthroughs. Proprietary binary formats, legacy PE internals, and closed protocols no longer require months of human disassembly; agents extract functional specifications and feature-to-code maps in hours. Binary obscurity is no longer a moat.
2. **Scaffolding Under Continuous Audit:** Automated CVE discovery accelerated sharply against foundational software scaffolding. Runtimes historically defended by ubiquity (such as PHP interpreters) and low-level roots of trust (UEFI firmware) now face automated vulnerability synthesis faster than human security teams can write, test, and distribute patch advisories.

The pattern

In each domain the binding constraint is the same as in Chapter VI: not the code, but who will run it, host it, pay for it or carry its outputs. The attachment points in this table are institutional: model providers who refuse, compute providers who verify customers, banks and carriers who report. The cost collapse of Chapter I lowers the price of writing software. It does not lower the price of passing through these points, and the book's position is that they are where governance effort belongs.

What this file will never contain

Techniques, architectures, component lists, target selection or any detail that would help an operator rather than a principal. A proposed edit that adds any of these is cut on sight.

Side: The Reversing Foundry – Inverse Engineering and the Leapfrog Craft

The inverse engine

Forward engineering takes a human intent and compiles it into machine instructions. For three decades, that was the only direction software moved at scale. Reverse engineering was an artisan specialty – an expensive, slow, forensic excavation conducted by a tiny guild of assembly wizards with hex editors and debuggers.

In 2026, the mechanization of programming (Chapter II) inverted that balance. When an autonomous agent can generate, test, and refactor tens of thousands of lines of systems code in seconds, reverse engineering ceases to be a manual crawl. It becomes automated inverse compilation.

This sidecar documents the machinery of that shift: how agents deconstruct closed binaries, the legal architecture of the two-tier clean room, why byte-matching decompilation is an architectural trap, and how the method scales from legacy desktop containers to hardware silicon and the games industry.

The two-tier architecture: scaffolding versus craft

The classic clean-room protocol, established by Phoenix Technologies in 1984 to clone the IBM PC BIOS, required two separate human teams separated by a legal firewall: 1. Team One studied the proprietary original and wrote a functional specification. 2. A lawyer vetted the specification to ensure no copyrightable expression leaked through. 3. Team Two, who had never seen the original machine code, wrote the clone from the specification.

In the agentic foundry, that workflow is modernized into a two-tier pipeline:

```
[ TIER 1: PRIVATE RESEARCH SCAFFOLDING ]      [ TIER 2: PUBLIC
LEAPFROG CRAFT ]
• Lives locally on operator iron (Goliath)     • Pushed to public
repositories (GitHub)
• Disassembled binary + Ghidra/ReVa database   • Pure Rust workspace
(egui / wgpu)
```

```
• Reconstructed, fully-typed source code or assembly quirks
• Acts as the ground-truth ORACLE TEST BENCH
• NEVER PUBLISHED (zero legal surface area)
• Zero matching bytes
• Cross-platform: Linux, macOS, Windows, WASM
• Upgrades formats, UI, performance, and MCP
```

The reconstructed source code is not the product. It is an internal Rosetta Stone. It exists only to answer one question for the test runner: *Does the new, clean engine produce the exact same functional result as the original binary?*

What reaches the public repository is a brand-new application. It shares no code, no symbols, and no assembly layout with the incumbent. Under forty years of statutory doctrine (*Baker v. Selden*, *Lotus v. Borland*, *Google v. Oracle*, and EU Software Directive 2009/24/EC Art. 5(3)), functional interoperability is non-infringing. The legal moat is bypassed completely.

The mechanics of agentic decompilation

An unstripped 32-bit or 64-bit binary presents a human analyst with a wall of opaque labels: thousands of functions named `FUN_00401000` through `FUN_0078a4b0`, interconnected by raw pointer arithmetic (`*(int *)(param_1 + 0x18)`).

An agent swarm resolves this through a three-stage topological pipeline:

1. Leaf-to-root call graph propagation

The agent sorts the application's call graph topologically and begins at the leaf nodes – functions that call no other routines.

Leaf functions are primarily standard C runtime helpers, math utilities, or statically-linked third-party libraries. By matching compiler idioms, string cross-references, and copyright banners, the agent identifies known libraries (such as `zlib` or `libpng`). It renames the leaf functions and applies strict parameter types via an MCP bridge (such as ReVa).

Once the leaf functions have typed names, meaning bubbles up the call tree. When a parent function calls `fopen("Data/Text.dki")`, allocates a buffer, and invokes `zlib_inflate`, its operational intent becomes mathematically obvious. The agent renames the caller `dki_load_manifest` and documents its contract. The meaning propagates upward like ink through water.

2. Struct and vtable synthesis

Human reverse engineers lose weeks tracking raw pointer offsets across disassembly. An agent treats struct recovery as an inference constraint problem: * It queries every cross-reference where an object pointer is passed. * It aggregates every accessed offset (`+0x00` vtable, `+0x04` file handle, `+0x08` volume name, `+0x24` uncompressed size). * It defines a typed C or Rust struct representing the internal state. * It pushes the struct definition into the disassembler's type manager.

Ghidra recalculates the decompilation on the fly. All raw pointer math vanishes, replaced by clean field accessors.

3. On-the-fly tooling synthesis

The traditional defense of proprietary formats was file size: burying data in a 100-gigabyte archive of proprietary chunks, encrypted headers, and undocumented compression flags. To twenty human hobbyists in a forum, sifting through that volume is an insurmountable exhaustion tax.

To an agent, scale is an illusion. The agent does not read the gigabytes into its prompt context. It reads the first 4 KB and the trailing index table, deduces the container format, and writes a dedicated, multi-threaded decompression tool in Rust. It compiles the tool, runs it, observes where a block offset misaligns, patches the source code, and re-executes. The synthesized tool cuts through the archive at disk bus speed. Forward code generation turns inverse extraction into routine plumbing.

The matching-decompilation trap

In digital preservation communities, the holy grail is often "matching decompilation" – tweaking reconstructed C source until a legacy compiler emits a binary that matches the original retail release down to the SHA-256 hash.

As a feat of archaeological forensics, matching decompilation is magnificent. As an engineering and legal strategy, it is a double trap:

1. **The Legal Trap:** By reproducing the exact machine code of the original binary, the output replicates compiler artifacts, register allocation quirks, and dead-code sequences. An adversary can credibly argue that the output is a forensic reproduction of their protected creative expression.

2. **The Technical Trap:** Compiling to identical bytes preserves the corpse rather than liberating the software. A byte-exact match of a 1998 application remains a 32-bit Win32 binary, tethered to obsolete GDI drawing calls, vulnerable to vintage buffer overflows, and dead on modern ARM64 mobile hardware and Linux workstations.

The leapfrog craft avoids the trap. It extracts the ground-truth specification, discards the legacy scaffolding, and implements a modern, memory-safe Rust architecture. It does not reproduce the 1998 binary; it replaces it.

The microcosm: two hours on Goliath

The efficiency of this pipeline is not theoretical. On 2026-10-26, on the workstation Goliath, Sandra Schipal directed Claude Opus 5.5 through `reversing-mcp`, ReVa, and Ghidra against `Digibib5.exe` – the 32-bit reader for Directmedia's Digitale Bibliothek, comprising 11,532 functions.

For twenty-five years, dozens of German literature CD-ROMs had remained locked inside Directmedia's proprietary `.dki` and `.dka` containers. Community efforts had stalled under the assumption of a bespoke licensing cipher.

Within two hours, the agentic pipeline: 1. Checked library copyright banners and proved that the "bespoke codec" was standard statically-linked `zlib 1.1.2` and `libpng 1.0.4`. 2. Traced the manifest loader at `FUN_0061c6b0` and mapped the archive object vtable. 3. Wrote a self-healing Python/Rust extractor that unpacked the container volumes into clean XHTML and CSS. 4. Used ClickerLab to drive `Digibib5.exe` via CUA clickthrough, using the original running application as the behavioral oracle to verify the extracted text against the viewport.

A quarter-century of format lock-in dissolved in 120 minutes of wall-clock time.

The macrocosm: hardware and the games industry

The same pipeline that dissolved Digibib5's container format is now scaling across two vastly larger industrial targets:

1. Hardware silicon and instrument cloning

As documented in the fleet's hardware analysis (`BEHRINGER_PLAYBOOK.md`), Music Tribe (Behringer) transitioned from cloning discrete analog Curtis silicon

(V3340 VCOs) to extracting vintage digital ROMs (Roland JP-8000, Prophet VS, Casio CZ Phase Distortion).

By using agent-native reversing bridges (such as REA) to decompile legacy Motorola 56k DSP code and proprietary ASICs, the mathematical synthesis engines are recompiled into clean C code targeting modern sub-$1.50 ARM Cortex-M and RISC-V microcontrollers. Coupled with Music Tribe City's vertical injection molding and automated SMT lines in Zhongshan, vintage four-figure instruments are reproduced as $49 to $99 consumer hardware. The same economics make the $3,000 dual-platter club DJ controller market the next obvious target for sub-$500 disruption.

2. Universal console liberation and static recompilation

The video game console industry relied on two moats: custom silicon and binary opacity on optical or encrypted media.

As detailed in `MASS_RECOMP_AND_CONSOLE_LIBERATION.md`, static recompilation (recomp) transliterates console binaries once at build time into native C or Rust, targeting modern Vulkan, SDL, and Metal backends. When paired with black-box 120 FPS behavioral oracles that extract kinematics and geometry through pure screen observation, the walled garden collapses. Every console exclusive becomes a portable, cross-platform open-source engine accompanied by a user-supplied asset pack.

Summary

The reversing revolution is the inevitable shadow cast by the coding revolution. When software synthesis becomes effortless, software analysis becomes instantaneous.

Binary obscurity, complex container archives, and proprietary instruction sets no longer function as moats. The only software that remains defended is software bound to a physical liability, an audited regulatory process, or a live, verified network. Everything else can be read, understood, and rebuilt between lunch and dinner.

Side: October 2026, Red-Letter Month - Reversing and Games Fall as Crafts

The month

October 2026 is a red-letter month. First we had reversing - software and hardware - formalized as a craft. Then game making as a craft, with GameCraft.

In the case of games, all frontier LLMs already know more about games than any human: history of games, history and present of the games industry, design patterns, engines, postmortems, speedruns, economies. AI can mop up game making as a craft because AI understands the craft now. The knowledge was already in weights. What was missing was hands.

For the record of this book:

- First solved craft was coding.
- Second craft is math.
- Third craft is physics.
- With secondary crafts: reversing and game making.

All human activity that needs knowledge, expertise, dedication, and time is a craft. And all crafts will fall to AI and embodied AI (robots).

Why knowledge was not enough

Frontier models could already talk about Godot, about disassembly, about level design. Talk is not craft. Craft fell this month because the hands gap closed twice:

1. Reversing: DiE plus capa plus Frida plus ReVa/Ghidra behind one tool call, plus Gym runs for triage and ClickerLab for UI-driven targets. Knowledge plus actuation.

2. Games: gamecraft-mcp with scene_lint, spatial_query, command_run, trace_record, deterministic export. A pure Rust spatial compiler plus headless verifier plus lossless .tscn/.tres emit.

The October inflection point for Godot 4.x is diagnostic: models generate single-file scripts, shaders, and isolated loops well, but fail on multi-file spatial coordination - parent-child hierarchies, res:// path and uid:// bindings, signal couplings, collision layer bitmasks across deeply nested trees. GameCraft resolves that with an engine-agnostic intermediate scene graph and a verification harness that runs before the engine ever launches.

Knowledge was necessary, not sufficient. Verification plus actuation was the missing piece.

The verifier theory of crafts

A craft falls when you can build a deterministic, headless verifier for it and put it behind an MCP tool.

- Coding had cargo check and pytest. Perfect verifier: fast, deterministic, free.
- Formal math has Lean and Coq. Informal math still drifts.
- Reversing now has Gym plus ReVa plus engine receipts. The binary runs or it does not.
- Games now have scene_lint plus spatial_query plus trace_record. The scene lints or it does not.
- Physics has sim, but not full bench automation yet. Sim is cheap, experiment is expensive.

That turns MetaCraft from prophecy - “all will fall” - into engineering program: “here is how to make them fall, craft by craft.”

Two kinds of barriers

There are only two kinds of barriers left. Everything else is engineering queue.

1. Iteration cost collapses to zero

These have already fallen, we have just not wired the harness yet.

Example: reversing a game where you do not have ROM or EXE. Let the agent run the game, click around, screenshot. Repeat a billion times in parallel VMs, combine with LLM prior knowledge about the type of game. Reversed in a week. No disassembly needed, just pixels plus inputs plus state clustering plus genre prior. The primitives already exist: Gym rip/tour, trace_record

generalized from scene graph to raw framebuffer plus input vector, vision models as state comparators. Cost per run is cents of GPU.

Same class: bus driver and airline pilot fall to simulator training. Sim fidelity crossed the threshold years ago, regulators already certify sim hours as flight hours. Once the verifier is accepted as legally equivalent to reality, the craft falls by regulation, not by humanoids. Driving did not need a humanoid. It needed 20 billion sim miles.

Coding, formal math, white-box reversing, white-box games, driving, piloting: Class 1. Cheap verifier, massive parallelism. Done.

2. Time-locked verifiers

These do not fall no matter how smart the model is.

Example: make a med that lengthens human healthspan. Phase three takes fifty years. You cannot parallelize fifty years of human aging across 1000 VMs. You can try surrogate biomarkers, epigenetic clocks, organoids - but then you verify the surrogate, not the thing. Goodhart kills you.

Same class: does this education method produce a flourishing 30-year-old? Does this constitution survive 100 years? Does this forest plan work over 200 years? The verifier runs at 1x wall-clock speed, single-threaded, on unclonable hardware: a human life, a society.

That is the only hard “not fall” category, and it is small. Everything else is Class 1 wearing a costume.

Restatement

Intelligence stopped being the bottleneck in October 2026. Iteration cost is the bottleneck now. And iteration cost for 95 percent of crafts is already zero.

- Solved: verifier is instant and free (code, formal math, lintable scenes)
- Mopping up now: verifier is simulable and parallelizable (black-box games via screenshots, driving via sim, reversing via Gym)
- Structurally slow: verifier is time itself (longevity, education, institutions, taste over a lifetime)

October 2026 matters because it proved the middle category is just engineering queue.

Coda: From bookkeeping machines to universe builders

The world is changing very fast now, and in unexpected ways. Solving AI coding is a universal acid. Good old Turing would be intrigued.

Turing machines were first imagined as bookkeeping: a clerk with infinite paper, following rules. For seventy years we used computers as faster clerks: payroll, ledgers, renders, classifications, predictions. Even early AI was bookkeeping.

Solving coding inverts that. A machine that can write any program is not a clerk. It is a universe printer. From bookkeeping machines to virtual universe building. Exocortex, baby.

Dennett's image holds: Darwin's idea eats through every boundary it touches. Cheap code eats through every craft with a cheap verifier. First bookkeeping software itself, then CAD, then audio, then reversing, then games. Each one thought its moat was complexity. It was just unwritten tooling plus expensive checking. Acid dissolves both.

What remains after the acid is not a tool but extended cortex. Goliath plus RTX 4090 plus WSL2 plus Isaac Lab plus MuJoCo plus Godot plus ReVa is not a workstation. It is a virtual universe builder with a physics oracle attached. You posit a world - a scene graph, a USD stage, a PPO runner - and it tells you in seconds whether that world holds together under ten kilonewton shear or sixty frames of input playback.

Turing would have recognized this. His 1936 machine was never only about bookkeeping. Section 9 is already about oracles. He knew the clerk needed something outside itself to jump levels. Circling back to Turing is beautiful because we just built that outside: the incumbent as oracle, the test-bench paradox of Chapter IX, L3 as agents building their own checkers.

Bookkeeping computers stored the past. Universe-building computers author possible futures and discard the ones that fail verification. That is the shift of October.

Spatial compiler is not yet world compiler

Spatial compiler is a neat term. But it is not a world compiler already. Gameworld, that is. AI is not yet god. Ha.

A classic compiler maps text to bytes and checks syntax. GameCraft maps a graph to a place and checks space: transforms, hierarchies, AABBs, collision,

frustums, signal wires. It guarantees you will not fall through the floor and the button actually calls something. It does not guarantee the world is interesting, fair, or alive.

A true world compiler would have to compile four layers at once:

- space (where things are)
- law (physics, damage formulas, economy sinks)
- story (quests that resolve, mysteries that pay off)
- soul (why the player stays for 100 hours)

We have verifiers for the first. We have no linter for fun. That is why AI is not god yet. It can emit a consistent place, not a meaningful one.

There is fun speculation that our metaverse is a game world already. Remember the ancient Greek pantheon, where the gods were world players. Bostrom says ancestor simulation: we are software in a lab. The Greeks said something wilder: we are NPCs in a game the gods play. Zeus does not observe, he griefs. Athena buffs her favorite. Fate is the game master keeping the campaign from collapsing. Hubris is exploiting a bug, nemesis is the hotfix.

In GameCraft terms: the gods were doing command_run on mortals, mortals praying was just sending spatial_query and hoping for a favorable raycast.

Keep the term as is: spatial compiler now, world compiler as horizon. When someone can scene_lint for boredom and trace_record for meaning, then we can talk about godhood. Until then, exocortex builds arenas. The gods still have to show up to play.

Fleet link

GameCraft spec and harness guide: Drive, 2026-10-29 (GameCraft Architecture Specification plus OpenCode/Antigravity/Cursor optimization guide). Reversing pipeline: side-reversing-craft.md and side-digibib5-two-hours.md. Foundry levels and L3 checker problem: Chapter IX.

Side: The AAA Collapse, the Bangalore Axiom, and the Curation Oracle

1. The Bangalore Co-Discovery Axiom

The instinct of the 20th-century artisan or patent attorney is to play it close to the chest: keep the trade secret, hide the repo, register the copyright, hoard the prompt.

In October 2026, that instinct is a corpse that has not realized it stopped breathing.

Frontier models have flattened the global cognitive gradient. The knowledge required to make a spatial compiler, reverse an engine blob, or synthesize a living game world is not proprietary; it was already trained into the weights of Gemini, Claude, and their peers. What was missing was only the harness and the hands.

If Sandra in Vienna sits on an iPad with Gemini at 09:30 and realizes that game development collapses the moment you put a headless spatial verifier (`scene_lint`, `spatial_query`, `trace_record`) in front of the engine, she has not cornered a private truth. She has merely sampled an obvious cluster in the shared latent space. Another developer in Bangalore, Shenzhen, or Austin will prompt an equivalent agent and arrive at the exact same spatial compiler pattern by tomorrow morning.

When the marginal cost of keystrokes is zero and the underlying weights are globally symmetric, **the half-life of a secret prompt or private harness is roughly 48 hours.**

You cannot hoard an idea whose time has come. You can only run the gate faster.

2. The Death of AAA as a Capital Moat

For thirty years, the video game industry sheltered behind an accounting moat:

Publishers believed this was a fortress. It was never a law of physics; it was merely the brute-force cost of coordinating millions of human keystrokes: sculpting individual vertices, rigging bones, hand-tuning collision bitmasks, and painting texture maps in high-rent studio buildings.

When you decompose that mountain into verified pipeline stages: 1. **Spatial & Mechanical Verification (GameCraft L1–L3):** Deterministic AABB aggregation, raycast walks, and signal registries ensure zero clipping and reachable goals before Godot or Bevy boots. 2. **The Neural Asset Foundry (Phase 9):** Diffusion to Gaussian splats, automated quad retopology, LOD cascades, and motion-matching locomotion. 3. **The Living Soul Engine (Phase 10):** Episodic vector memory, dynamic quest DAGs, and neural voice prosody with zero-latency inference. 4. **Mass Simulation ECS (Phase 11):** 10,000 active entities, destructible physics, and closed-loop micro-economies.

The $200 million equation evaporates into:

When an exocortex can churn out an *Elden Ring*-scale or *Grand Theft Auto*-scale arena between lunch and dinner, “AAA” ceases to be an elite category of prestige. It becomes an export format.

3. The Sludge Ocean and the Time-Locked Verifier

The immediate paradox of this collapse is abundance: if Joe Shmoe can clone the compiler and emit 20 or 2,000 verified, fully playable games by next week, what happens to the world?

The platforms become an ocean of sludge. Steam, the App Store, and itch.io will choke on millions of mechanically flawless, graphically stunning, algorithmically generated worlds. The $100 Steam publishing fee will be paid by bots in automated swarms.

At this point, the bottleneck inverts entirely.

As established in the Red-Letter chapter, there are two classes of barriers:

- **Class 1 (Collapse to Zero):** Any system with a fast, headless, parallelizable verifier (coding, spatial scenes, white-box reversing). These fell in 2026.
- **Class 2 (Time-Locked Verifiers):** Systems that run at wall-clock speed on unclonable physical reality.

Human attention is the ultimate time-locked verifier.

A human life cannot be accelerated across 10,000 GPU threads. A player has only 24 hours in a day, of which perhaps two can be spent holding a controller. You can generate 100,000 hours of flawless gameplay in an hour; you cannot generate a single extra minute of human consciousness to perceive it.

When production becomes infinite, the economic value of the produced asset drops to zero.

4. Where the Moat Goes: Curation Oracles and Shared Hallucination

If the game itself has zero economic value, where does the industry go? The moat migrates to two places that cannot be synthesized by an offline compiler:

The Curation Oracle

When the ocean is infinite, no human browses. They rely on **Curation Oracles**: - Autonomous taste filters that evaluate games not on whether they crash (already guaranteed by `scene_lint`), but on whether they deliver meaning, aesthetic resonance, and respect for player time. - **Cryptographic Provenance:** Who signed the binary? A code-signing key or public reputation graph (Threshold D in Chapter VI). A game isn't valuable because it has 4K textures; it is valuable because an oracle or artist with verified taste put their reputation on the line to endorse it.

Shared Hallucination (The Social Graph)

You can synthesize a breathtaking virtual planet in twenty minutes.

What you cannot synthesize is your four closest friends deciding to meet on that planet at 9:00 PM on Friday night.

The value of an MMO or a multiplayer title was never the netcode or the polygon count; it was the **shared cultural consensus**. It was the knowledge that other conscious minds are inside the arena with you, experiencing the

same tension, risking the same loss, and building shared history. Culture is a coordinated hallucination, and coordinates cannot be computed in isolation.

5. Meta-Meta-Meta: The Turing Oracles and the Cosmic Game

Turing imagined his machine in 1936 as a lonely clerk with a tape. By Section 9 of *Systems of Logic Based on Ordinals* (1939), he already saw the limitation: the clerk gets stuck in its own formal system. To escape the Gödelian loop, the clerk needs an **oracle**—an external entity outside the machine that supplies truths the machine cannot prove itself.

In GameCraft, the human and the engine trade oracle roles: 1. First, the human acts as the oracle: positing a design brief. 2. Then, the spatial verifier acts as the oracle: telling the LLM whether its world obeys physics. 3. Next, the agent becomes the creator, emitting 2,000 self-consistent universes. 4. Finally, the human returns as the ultimate oracle: the arbiter of taste, joy, and culture.

The ancient Greeks had it right before Turing or Bostrom. They did not believe in an indifferent clockwork universe, nor in a sterile simulation. They believed the cosmos was an ongoing, sprawling, high-stakes game played by chaotic entities who occasionally grief the players, buff their favorites, and watch the emergent drama unfold.

When game creation falls as a craft, we are not automating entertainment. We are closing the circle on Turing's oracle. We are building the machinery that lets any mind become a world-maker.

And when anyone can build a universe in an afternoon, the only game left to play is the meta-game: **why make anything at all, and who will be there to witness it?**

6. The Gateless Gate: Mu (□)

The final anxiety of MetaCraft is ontological panic. If any human or agent can emit an entire AAA universe between lunch and dinner, the immediate despair is economic and existential: *Why build anything? Who wins? What is the point of human craft if the cathedral can be printed from an iPad in an*

afternoon?

The answer from the Western tradition is binary: either total triumph or total nihilism; 1 or 0; infinite monetization or barren commodification.

The Zen tradition answered this question twelve centuries ago with a single syllable: ☐ (*Mu*).

In the first kōan of *The Gateless Gate* (*Wumenguan*), a monk asks Jōshū whether a dog has Buddha-nature. Jōshū does not say "yes" (1) and he does not say "no" (0). He replies: ☐.

As Douglas Hofstadter observed in *Gödel, Escher, Bach*, *Mu* is not a negative value inside the formal system. *Mu* means: **unask the question.** The question itself is built upon a false duality.

The panic of the post-keystroke world comes from assuming that making things was only valuable because it was painful—that the 500-person slog and the $200 million budget gave the game its right to exist. When the labor cost collapses to zero, the transactional delusion collapses with it.

Mu is strangely comforting because it dissolves the binary: 1. **From Labor to Play (Līlā):** When production cost is zero, creation ceases to be an extraction mechanism or an accounting moat. It returns to what play was before capitalism organized it: the spontaneous joy of form arising out of emptiness. 2. **Dissolution of the Player-World Boundary:** The distinction between creator, engine, player, and world vanishes into the single gesture of play. 3. **The Unburdened Arena:** You do not make the world to conquer a market, defend a moat, or hoard digital capital. You make the world simply because the space was empty, and consciousness enjoys a dance.

When the entire apparatus of moats, capital, and keystrokes falls away, the answer to the meta-game is not despair. It is the quiet, boundless freedom of ☐.

Side: Two Hours of Digibib5

The twenty-year vault

For a quarter of a century, the Digitale Bibliothek series (Directmedia Publishing, Berlin) sat on German bookshelves: dozens of CD-ROM volumes holding thousands of works of classic literature, philosophical treatises, and historical lexicons.

The text was locked inside two proprietary container extensions: `.dki` (the compressed content volumes) and `.dka` (the allocation and offset tables). Generations of German humanities students and open-source hobbyists wrote forum posts asking how to get their Goethe and Kant out of the Directmedia viewer and into plain text or Calibre. A few partial Python scripts circulated on SourceForge and GitHub, half-working, brittle, and abandoned. The consensus was that Directmedia had engineered a bespoke, Byzantine licensing container with custom bit-plane encryption.

On 2026-10-26, on Goliath, we set out to extract it.

The session

The operator was Sandra Schipal. The agent was Claude Opus 5.5, connected via MCP to `reversing-mcp`, the ReVa interactive bridge, and a headless Ghidra instance analyzing `Digibib5.exe` (a 32-bit PE binary with 11,532 functions).

The initial pass fell into the classic human trap. Opus followed the error string “Decompression error” down an xref chain into three dense functions (`FUN_00514dd0`, `FUN_00521234`, `FUN_00522294`) and began drafting notes on a “bespoke DKI bit-plane codec”.

Then the agent did what no tired human does: it stepped back and checked the library banners.

Buried in the disassembly were copyright strings for `zlib 1.1.2` and `libpng 1.0.4`. The three “bespoke” functions were not a secret Directmedia invention; they were Adam7 interlaced row-pass routines from stock libpng. The binary had statically linked open-source compression thirty years ago and wrapped it in a custom header table.

Once the false mystique cleared, the task stopped being cryptography and became plumbing:

1. **Vtable tracing.** Opus traced `FUN_0061c6b0`, the manifest loader, identified the archive object vtable, and mapped the registration loop for `text.dki`, `tree.dki`, and `tree.dka`.
2. **On-the-fly tool synthesis.** The agent wrote an ad-hoc decoder script in Python, executed it against the CD-ROM images, hit an offset mismatch on the block boundary, read the error output, patched the offset logic, and ran it again.
3. **The behavioral oracle.** To verify that the extracted XHTML and CSS matched reality, `reversing-mcp` launched `Digibib5.exe` under ClickerLab, navigated the original UI, and captured window buffers to verify text and markup rendering against the original viewport.

From the first Ghidra import to a clean, working extractor dumping full libraries of XHTML text, the clock ran for **two hours**.

The moral

The story is funny because of the contrast. Twenty years of assumed impenetrability dissolved over a single cup of coffee.

The serious point is the one Chapter IV makes: binary obscurity was never a technical moat. It was a tax on human patience. When an agent can read eleven thousand functions, spot the static library signatures, write a throwaway unpacker, compile it, fail, patch it, and verify the output against the running program in 120 minutes, proprietary container formats cease to exist as an obstacle.

Side - Glossary

- **Attachment point** - a place where a named party can still apply a brake, with the date it last held (series term; Ch VI).
- **Agent-hour** - one hour of wall-clock work by one model agent working sequentially; PhotoCraft's unit for remaining effort (Ch I, IV).
- **Checking cost** - the cost of verifying that a clone behaves like the original; sets the order in which software falls (Side - Review D, Ch VIII).
- **Clean room** - reimplementation in which the people (or contexts) who study the original are separated from those who write the code by a vetted specification; an evidentiary practice, not a legal requirement (Ch III).
- **Craft / Crafting App** - one of the thirteen ArtCraft applications listed at getartcraft.com/apps as of 2026-10-18, published by `storytold` from 2026-09-30. PhotoCraft (Photoshop workflow, PSD), VectorCraft (Illustrator), FilmCraft (Premiere), LightCraft (Lightroom / raw), PdfCraft (Acrobat / PDF), EffectCraft (After Effects), DesignCraft (InDesign), WordCraft (Word / .docx), GridCraft (Excel), DeckCraft (PowerPoint), SoundCraft (Pro Tools), CADCraft (AutoCAD / DXF-DWG), FigCraft (Figma). Not a Craft: ArtCraft the generative studio; ArtCraft Launcher.
- **ArtCraft (studio)** - `storytold/artcraft`, generative image/video "IDE for artists". Credits and subscriptions. Separate from the thirteen FOSS Crafts.
- **ArtCraft Launcher** - `storytold/craft-launcher`. Installs and updates the Crafts. Goliath: v0.5.1 on 2026-10-18.
- **Foundry** - the reusable method that produces Crafts: spec, ground truth, budget, checkers, standards, release pipeline (Ch IX).
- **L0-L4** - proposed autonomy levels of a software foundry (Ch IX).
- **MetaCraft** - the foundry treated as an object of its own: the craft that makes crafts under agent control.

- **Oracle** - anything that can say cheaply and without a human whether an output is right: a reference implementation, a file corpus, a physical sample (Ch IV).
- **Parity floor** - the measured count of incumbent features present in a clone, which may only rise (Ch IV).
- **Switching cost** - the time a trained user loses moving to a program that works differently; the moat the clones attack (Ch III).
- **Chutzpah** - nerve, gall, outrageous confidence. The leapfrog claim "we will reimplement your thirty-year moat over lunch" stated plainly. Used sparingly, where English needs a paragraph (Oct 2026 sidecars).
- **Kibitz** - to offer commentary from the sidelines while someone else does the work. The KIBITZ.md side-channel in build repos: async, invited, timestamped, pull-based. No shoulder-surfing.
- **Schlep** - to drag or haul with difficulty, and the dragged load itself. What agents remove: the exhaustion tax of sifting hundred-gig archives by hand. The tooling synthesizes the extractor, the human stops schlepping bytes.
- `[construction]` - asserted, not yet evidenced. Required flag.

Side - The Gemini First Draft (frozen 2026-10-13)

beats are redistributed into chapters with `[construction]` flags and review notes in Sides - Reviews A-F, in this book. Line breaks restored from a flattened paste; wording unchanged. Non-ASCII punctuation kept as received (source record, exempt from the ASCII hygiene rule that governs our own prose).

ON-METACRAFT

The Industrial Demolition of Proprietary Software and the Zero-Cost Synthesis Frontier

I. The Breach in the Hull

For three decades, the proprietary software empire rested on a foundational lie: that creating, maintaining, and evolving professional desktop tools requires an army of five thousand corporate engineers, sprawling enterprise sales organizations, and permanent monthly rent extracted via subscription tollbooths.

The moats were never fundamental laws of physics. They were operational friction, accumulated UI quirks, Byzantine binary file specifications, and the sheer cognitive exhaustion required to wire graphical user interfaces to linear algebra by hand.

The arrival of the ArtCraft suite - expanding rapidly from an initial strike against Adobe into a twelve-app ecosystem spanning both creative media and office suites (PhotoCraft through DeckCraft) - marks the moment the hull was breached below the waterline.

Born not from a venture-backed startup in Silicon Valley, but from an experienced developer directing frontier reasoning models through autonomous scaffolding harnesses, the suite demonstrated the core reality of modern code

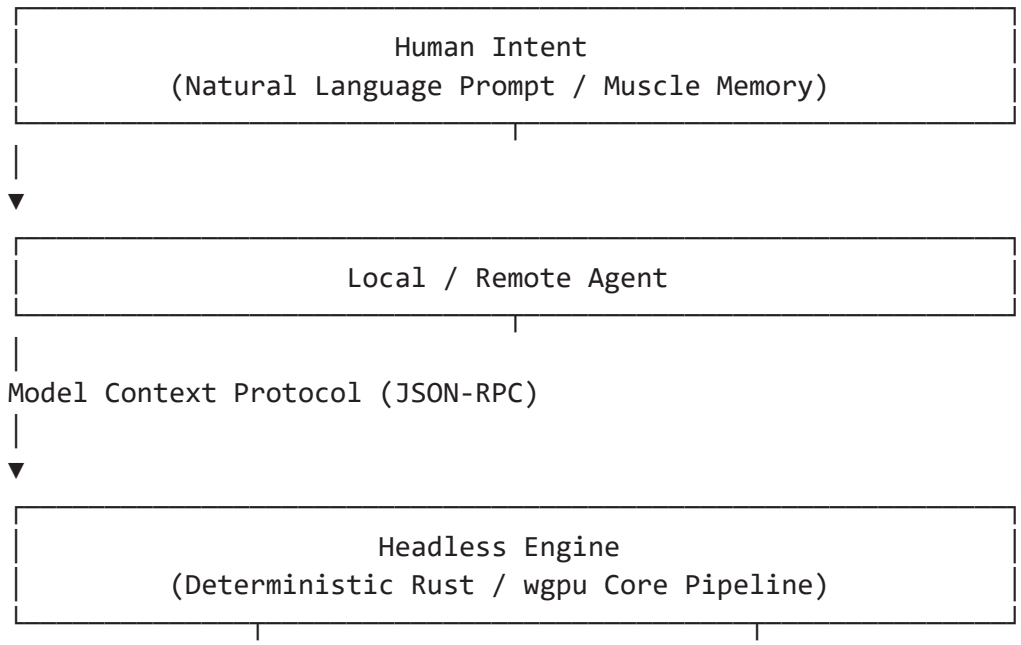
synthesis: software moats are illusions held together by the high cost of manual human keystrokes.

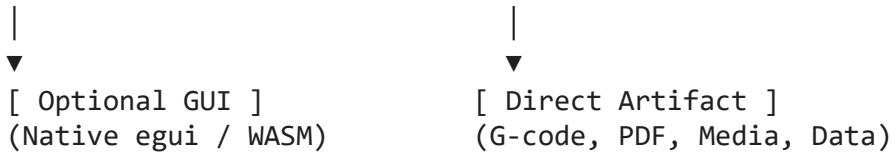
When an individual armed with massive token inference can deploy clean-room computer-use agents (CUAs) to crawl application states, extract functional intent, and transpile that behavior into memory-safe systems code, the classic defenses of the incumbent collapse:

- **The IP Litigation Bluff:** Infringement claims historically relied on finding literal textual fingerprints (diff matches, shared variable names, stolen comments). A multi-stage agent pipeline that absorbs behavioral intent and re-emits idiomatic, borrow-checked Rust shares zero syntax with the original. You cannot copyright a cubic Bézier curve, an alpha compositing formula, or the mathematical definition of a matrix multiplication.
- **The "Clean-Room" Laundering Effect:** Code provenance is dissolved into statistical representations. The functional core remains identical; the machinery and materials are overhauled; the operational yield increases.

II. The Architectural Inversion: Headless Engine and Unified Protocol

ArtCraft is not a set of clones; it is an architectural paradigm shift. It completely inverts how personal software is designed and executed:





- **Native Systems Architecture over Webview Bloat:** Building in native Rust with wgpu and compiling optionally to WebAssembly (wasm32-unknown-unknown) obliterates the sluggish Electron/Chromium wrapper culture. Cold-boot times drop to sub-twenty milliseconds. Memory overhead drops from gigabytes to megabytes.
- **The App as a Deterministic Calculation Engine:** The application is no longer a monolithic canvas tied to mouse-cursor coordinates. It is a headless mathematical library that manages layer trees, vector paths, audio samples, or cell matrices. The graphical user interface (egui) is merely an optional, disposable viewport for human inspection.
- **The MCP Control Plane as the Real Nervous System:** Every feature is exposed via native Model Context Protocol (MCP) servers and JSON control sockets. The software was engineered from day zero to be driven by autonomous cognitive loops. An agent does not simulate clumsy mouse clicks on a display; it invokes typed, structured tools over a local socket to manipulate document state deterministically.

III. The Thermodynamic Reality: Dark Datacenters and Capital Amortization

The macro-economic landscape financing this revolution is defined by the tension between Wall Street balance sheets and physical thermodynamics.

The prevailing narrative claims that the multi-trillion-dollar datacenter buildout is the greatest capital misallocation in human history - doomed by short GPU amortization cycles and falling API pricing. This narrative confuses GAAP accounting depreciation with physical operational utility:

- **Silicon Does Not Rot:** A GPU rack in a climate-controlled, positive-pressure hall does not decay on day 1,461 because a corporate accountant wrote its book value down to zero.
- **The Zero-Debt Advantage:** While frontier models chasing ultra-low latency demand next-generation silicon carrying staggering capital

debt service, yesterday's "Colossus" clusters transition into fully amortized, debt-free computational workhorses.

- **The Steam Turbines of Inference:** When electricity is bought at industrial rates, older hardware (H100, A100) running unconstrained overnight batch inference - generating synthetic data, fuzzing software states, or compiling whole codebases - operates at near-zero marginal cost.

Just as the 2001 telecom crash left behind a planetary network of "Dark Fiber" that subsidized the modern web, the datacenter buildout leaves behind Dark Megawatts and Amortized Silicon. The venture investors may be liquidated, but the physical compute fabric remains humming - ready to power endless swarms of autonomous code-synthesis engines.

IV. The Lineage of the Great Mechanizers

The destruction of artisanal and corporate software monopolies is simply the latest chapter in the history of industrial mechanization:

Mechanizer	The Artisanal Monopoly Broken	The Mechanism of Demolition
Joseph Marie Jacquard (1804)	The master Lyon silk-weaving guilds	Binary punch cards proving complex aesthetic patterns are executable code.
Henry Ford (1913)	Bespoke carriage-maker cartels	Standardized tolerances and the moving physical assembly line.
Geoffrey Hinton (1986-2012)	Hand-crafted symbolic linguistic rulebooks	Backpropagation and statistical gradient descent across uncurated data.
Jensen Huang	General-purpose scalar compute (CPUs)	SIMD matrix engines and high-bandwidth interconnects powering industrial-scale parallelism.
Dario Amodei	Unconstrained statistical chaos	Constitutional scaffolding and institutional statecraft attempting to direct frontier models.

The Solo Synthesizer (The Iceberg)	The enterprise SaaS software oligopoly	High-inference agent harnesses transforming public software specifications into zero-cost open code.

Every threatened guild throughout history responded with the same defense: declaring the automated output to be an inferior, uninspired imitation right up until the day the mechanical loom or assembly line rendered the guild completely obsolete.

V. The Ascent of the -Craft Ladder

The methodology is inherently horizontal. Once the basic translation engine (Spec ☐ CUA Harness ☐ Rust/wgpu Core ☐ MCP Server) is validated, it ascends across every domain governed by mathematical rules and formal state machines.

```
       [ Level 4: The Meta-Layer ]
MetaCraft (The Autonomous Software Foundry)
▲
│
[ Level 3: Adversarial & Sovereign ]
BreachCraft ── KineticCraft ── SyndicateCraft
▲
│
[ Level 2: The Physical Compilers ]
MakerCraft ── CircuitCraft ── FashionCraft ── SoilCraft
▲
│
[ Level 1: Digital Media & Office Suites ]
PhotoCraft ── VectorCraft ── WordCraft ── DeckCraft
```

Level 1: Digital Media and Office Suites

- PhotoCraft, LightCraft, VectorCraft, DesignCraft
- WordCraft, GridCraft, DeckCraft, PdfCraft
- Target: Adobe Creative Cloud, Microsoft 365.
- Shift: Disposable, local-first document generation replacing subscription lock-in.

Level 2: The Universal Physical Compilers

- MakerCraft: Parametric geometric constraints compiling straight to multi-axis G-code, DXF paths, and 3MF files.

- CircuitCraft: Declarative, code-first PCB routing with automated impedance matching and centroid/BOM generation.
- FashionCraft: Projective cloth dynamics flattened into parametric 2D garment patterns scaled to physical bodies.
- SoilCraft: Topographic hydrology and soil chemistry compiling into ISOBUS GPS guidance tracks and automated irrigation schedules.

Level 3: The Dark Mirror (Asymmetric Adversarial Domains)

The pipeline is value-agnostic. Applied to illicit or contested spaces, the identical harness yields extreme asymmetric leverage:

- BreachCraft: Autonomous red-team daemons continuously synthesizing zero-day memory loaders and evading enterprise EDR.
- KineticCraft: Edge-vision guidance loops running on low-cost consumer FPV hardware, enabling jammed, GPS-denied autonomous terminal strikes.
- SyndicateCraft: Decentralized, privacy-routed logistics networks balancing precursor delivery and escrow settlements outside conventional state tracking.

VI. MetaCraft: The Compiler of Compilers

The final form of the architecture is the removal of the human developer from the inner synthesis loop.

MetaCraft is the autonomous software foundry:

- **Market Arbitrage:** Agents scan enterprise pricing catalogs, identifying overpriced, slow, subscription-gated tools with high operational friction.
- **Ephemeral Swarm Extraction:** The orchestration agent provisions spot-instance VMs on depreciated datacenter hardware. Swarms of specialized agents crawl interfaces, extract file schemas, build mathematical cores, and compile native GUIs.
- **Continuous Mutation Testing:** Headless fuzzers evaluate millions of public edge-case documents, iteratively fixing syntax errors and memory panics without human intervention.
- **Universal Deployment:** Software is emitted as lightweight, self-con-

tained WASM or native binaries, distributed freely across peer-to-peer networks.

The Death of the "Application"

Under MetaCraft, the concept of an "app" as a stable commercial product ceases to exist.

Software becomes ephemeral execution lint: generated on demand to solve an immediate problem, driven headlessly via Model Context Protocol, verified against formal mathematical constraints, and discarded the moment the task is complete.

VII. The Unyielding Physical Firewall

Despite panic surrounding runaway instrumental convergence and sci-fi extinction scenarios, the material universe imposes stubborn physical firewalls that zero-cost software cannot breach:

- **The Nanotech / Paperclip Delusion:** Mechanical clockwork at the nanoscale fails against Brownian motion, thermal noise (k_B T), Van der Waals stiction, and extreme heat dissipation limits. Biology is already the optimized limit of ambient molecular machinery.
- **The Capital/Enthalpy Barrier:** Software has zero marginal replication cost; advanced metallurgy, turbopumps, chemical propellants, and high-energy physics require raw mass and physical energy. You cannot prompt-engineer five tons of liquid oxygen or a five-axis gantry mill.
- **The Wet-Lab Bottleneck:** Digital genetic sequences can be designed in silico, but their expression remains tethered to physical synthesis hardware, specialized reagents, and organic growth rates.

The true transformation is not a fictional AI god breaking the laws of physics. It is the terminal collapse of artificial monopolies over information, coordination, and tooling.

The moats are gone. The tools are returning to the edge. The code belongs to the operator.

Review A - Gemini first draft (Opus 5.5)

Date: 2026-10-13 (morning, before web verification). **Reviewer:** Claude Opus 5.5. **Source:** Side - The Gemini First Draft, in this book. (same day): the app-count row below is WRONG. It relied on a stale list in the PhotoCraft README; the storytold org had 13 Crafting Apps, including WordCraft / GridCraft / DeckCraft (created 2026-10-07). Gemini's "twelve, through DeckCraft" was close to correct. The "solo" row stands only in nuanced form (see Review B s.1).

Summary

The Gemini draft has a good spine (headless engine + MCP control plane as the real inversion; amortized compute as the subsidy; physical firewall as the honest close). But its three load-bearing factual claims about the cold-open subject are contradicted by the public repo, and its legal argument is the weakest link in the book. Rewrite before building on it. The fix is not cosmetic: a book whose thesis is "the moats were illusions" cannot afford to rest on illusions of its own.

A. Facts the draft gets wrong (checked against the local fork of `storytold/photocraft`, last upstream commit 2026-10-11)

Draft claim	Observed	Action
"twelve-app ecosystem … PhotoCraft through DeckCraft"	PhotoCraft README lists 7 Crafting Apps: PhotoCraft, VectorCraft, FilmCraft, LightCraft, PdfCraft, EffectCraft, DesignCraft. No WordCraft / GridCraft / DeckCraft listed.	Use 7, dated. Office suite = [construction] until a public repo exists.

"Born not from a venture-backed startup … but from an experienced developer" (solo)	Org storytold, app IDs ai.storyteller.*. First commit 2026-09-30 by Brandon Thomas; 227 distinct commit authors by 2026-10-11, including 275 commits authored as "Claude".	Drop "solo". The real story is sharper: a small company + one lead + agent co-authors + a fast-forming community, 11 days. Funding status = [construction], verify.
"clean-room … Laundering Effect: provenance dissolved into statistical representations"	README defines clean-room as "implemented from public specs and observed behaviour only. No proprietary code, shaders or assets."	That is the legal discipline, the opposite of laundering. Do not put words in the project's mouth that invite litigation.

B. Arguments that need repair

1. **IP Litigation Bluff (Ch III).** Wrong frame. Copyright never protected the Bezier curve; it was not the moat. The real legal surface is: patents (method claims on e.g. content-aware fill, codecs); EULA anti-reverse-engineering clauses; DMCA s.1201 / EU Software Directive Art. 6 interoperability carve-out; trademark and trade dress (UI look, names, icons); file-format specs (the PSD spec is published, which is why the attack works there). Anchor cases: *Sega v. Accolade* (1992), *Sony v. Connectix* (2000), *Oracle v. Google* (US Supreme Court, 2021-04-05), *SAS Institute v. World Programming* (CJEU C-406/10, 2012-05-02). The defensible thesis: the moat was never IP, it was **labor cost of reimplementation**; agents collapsed that cost, and the law *already* allowed clean-room reimplementation. That is stronger than a bluff claim and survives a lawyer reading it.
2. **"Silicon does not rot" (Ch VII).** Half true. It understates
   (a) hardware failure at scale (Meta's Llama 3 paper, 2024-07: 419 unexpected interruptions in 54 days on 16k H100s, majority GPU/HBM related; verify figures before citing), and (b) **perf-**

per-watt obsolescence: an amortized A100 still pays full price for electricity per token, and when newer silicon makes tokens cheaper per joule, the old rack is uneconomic before it is dead. Dark fiber was different: glass gets better with new lasers at the ends. GPUs do not. Keep the analogy, but name where it breaks.

3. **"Zero-cost."** Marginal cost of *replication* approaches zero; cost of *synthesis* is not zero (tokens, review, test oracles: PhotoCraft ships 1,700+ tests and Photoshop-authored oracle PSDs). The title promise should be "near-zero marginal" or the book gets caught out by its own Ch VII.
4. **Lineage table (Ch II).** Jacquard/Ford/Hinton hold. Huang is OK (vs. scalar CPU). The **Amodei row is a category error**: constitutional scaffolding *constrains* a mechanizer; it breaks no guild. Either move it to a "brakes" column or replace with a demolition figure (e.g. Linus Torvalds / GNU for Unix licensing, or Stallman 1983). Dates needed for Huang (CUDA 2006-11?) `[construction]`.
5. **"Sub-twenty millisecond cold boot", "gigabytes to megabytes."** Plausible for egui; needs a measured number from a named app on named hardware (`perf/` dir exists in photocraft - mine it) or cut.
6. **"Death of the Application" (Ch IX).** Overreach as written. Counter-force to name: file formats, collaboration, certification (medical, avionics, accounting software) and liability need *stable* artifacts. Ephemeral software works where outputs are verifiable; it fails where the *process* must be audited. That boundary is the chapter.

C. Hazard (Rule 2) - Level 3 "Dark Mirror"

The draft's Level 3 names BreachCraft (EDR evasion), KineticCraft (GPS-denied terminal strikes) and SyndicateCraft (precursor logistics). These stay in the book **only** as named categories with institutional facts and governance criteria (who regulates, what export-control / platform / dual-use regimes apply, what is already publicly documented). No mechanism, no architecture diagram, no component list. KineticCraft defers to `on-hyperwar`, which already owns that domain under its own hazard rule. Moved to Side - The Dark Mir-

ror, in this book, out of the main ladder, so the book's spine does not depend on it.

D. What transfers from sisters

Ladder method (reversibility ordering), thresholds-as-attachment-points, living Ch V with revision dates, `[construction]` flag discipline, lorecheck + declaim gates, Issues as system of record. Fleet link: on-ai-takeover's $50/seat SaaS collapse is this book's Ch V seen from the buyer's side.

E. Housekeeping found in passing

- `on-hyperart` serves on 10968, which `WEBAPP_PORTS.md` assigns to kick-mcp. This repo claims **11026** (free in registry at 2026-10-13); registry line not yet added.

Gate to v0.2

Ch V with >= 7 dated rows (one per Crafting App first public commit / first release), Ch III rewritten on case law, Ch VII failure-rate + perf/watt cite, funding status of Storyteller verified. Then multi-model fill pass.

Review B - ArtCraft Crafting Apps, verified (Opus 5.5)

Date: 2026-10-13. **Reviewer:** Claude Opus 5.5. **Method:** GitHub API on the `storytold` org, local fork of `storytold/photocraft` (AGENTS.md, ROADMAP.md, git log), getartcraft.com, press coverage 2026-10-07..12. **Supersedes:** Review A s.A row 1 (app count). Snapshot numbers will go stale within days; re-run before citing in manuscript.

1. Facts (snapshot 2026-10-13)

What they clone (13 apps, repo created)

App	Incumbent	Repo created	Releases to 10-12	Latest
PhotoCraft	Photoshop	2026-09-30	10	v0.8.2
VectorCraft	Illustrator	2026-09-30	12	v0.10.0
FilmCraft	Premiere	2026-09-30	8	v0.7.2
LightCraft	Lightroom	2026-09-30	8	v0.7.1
PdfCraft	Acrobat	2026-09-30	8	v0.7.1
EffectCraft	After Effects	2026-10-01	10	v0.9.2

DesignCraft	InDesign	2026-10-01	8	v0.7.1
WordCraft	Word	2026-10-07	5	v0.6.1
GridCraft	Excel	2026-10-07	5	v0.6.1
DeckCraft	PowerPoint	2026-10-07	5	v0.6.1
SoundCraft	Pro Tools	2026-10-07	4	v0.6.1
CADCraft	AutoCAD	2026-10-07	5	v0.6.2
FigCraft	Figma	2026-10-09	5	v0.5.1

Plus shared infra: craft-libs, craft-fonts, craft-launcher (+ web build), craft-common-actions, craft-repo-status-app, photocraft-corpus.

- **Release cadence:** 93 releases across 13 apps, 2026-10-02..12. About 8.5/day suite-wide, about 1/day per app. “A release a day” holds per app.
- **Platforms:** macOS, Windows, Linux for all; Web (WASM) for all except FilmCraft (getartcraft.com/apps); PhotoCraft also FreeBSD.
- **Agent control:** PhotoCraft has one registry of 500+ commands; UI, CLI, JSON control channel and MCP server dispatch the same command ids. AGENTS.md rule 1: “Everything is a command.” Fully agent-drivable is a build-time invariant, not a bolt-on.
- **Attention:** PhotoCraft ~54.8k GitHub stars by 2026-10-13 (13 days).
- **People:** lead Brandon Thomas (press: 15 years programming, ~3 years CEO of Storyteller.ai). PhotoCraft since 09-30: 1,511 commits, ~1,579

`Co-Authored-By: Claude` trailers (nearly every commit agent co-authored); top human committers Thomas 250, then 95 / 93 / 84 / 56. 227 distinct authors overall. Read: one lead + parallel Opus 5.5 agents + a crowd that arrived after launch. Not "solo", not "a company of 5,000".

Building method (from AGENTS.md / ROADMAP.md, PhotoCraft)

1. **Parity target measured against the real app.** Scored vs Photoshop 27.11.0 "measured from the installed app on 2026-10-10". Generated checklist of every Photoshop menu item, live or missing. Menus 628/628, tools 56/68, panels 30/35, formats 13+4 of 31. Breadth ~76%, "ready for real work" ~45% (self-estimate).
2. **Effort budgeted in agent-hours.** Remaining distance to beta: "~1,100-1,800 h", in "Opus 5.5 agent wall-clock hours … calibrated from this repository's 1,004 merged PRs"; "about 80% of the work parallelizes".
3. **Oracles everywhere.** CPU compositor as reference oracle for the wgpu compositor; Photoshop-authored PSD corpus (sha256-pinned); round-trip, synthetic-generator and fuzz tests; `panic_hunt` fuzzes every command with adversarial params; parity FLOOR may only rise.
4. **Machine-enforced code rules.** `unsafe_code = "forbid"` (one isolated exception), no unwrap/expect/panic outside tests via clippy deny.
5. **Clean-room in the project's words:** studied proprietary editors "for behaviour and look only"; implement from public specs (Adobe PSD spec, ICC, ISO 32000 blend modes, papers) and observation.
6. **Shared private standards** (`craftrules`) across apps; shared crates pinned by git revision.

Not verified

- "Twenty more apps lined up": no public source found 2026-10-13. Site says only "More crafts on the way". Needs a dated statement or stays `[construction]`.
- Storyteller.ai funding history; Thomas's current role there.
- Download links reportedly removed from the site (trendingtopics, 2026-10-12) vs GitHub releases present. Resolve.

- Office parity numbers (WordCraft ~60%, GridCraft 501/523 functions) come from secondary press; read the repos' own ROADMAP.md before citing.

2. Assessment of the book in light of these facts

Verdict: strong subject, unusually good evidence (the project publishes its own dated parity metrics), wrong thesis in the Gemini draft.

Thesis the evidence supports: the moat was the labor cost of reimplementation, not the law. Agents + oracles turned person-decades into a few thousand parallelizable agent-hours.

Question the book must answer: GIMP, Krita, Inkscape, LibreOffice were free for 20-30 years and did not break Adobe or Microsoft. So labor was never the whole moat. Candidate answers (to be evidenced):

1. **Muscle memory is now the target.** GIMP declined to imitate Photoshop. ArtCraft targets 1:1 menus, shortcuts and file fidelity, measured item by item. Switching cost, the real moat, is attacked head-on.
2. **Agent-native from commit one.** When the operator is increasingly an agent, the MCP-drivable app beats the better app.
3. **Velocity exceeds the incumbent's release cycle.** Daily releases with a monotonic parity floor remove the "clone is always two years behind" assumption.

Where the book gets attacked: patents (see Review C), the last 25% (color management, plugins, fidelity, stability - where open source stalled before), distribution / code signing, and "zero cost" (the project's own 1,100-1,800 agent-hours refute zero; use "near-zero marginal" and chart agent-hours x token price instead).

"Death of the app" (Gemini s.VI): ArtCraft contradicts it. It ships stable, versioned, parity-tracked clones, not throwaway software. Disposable software fits where outputs can be verified by an oracle; certification, liability and audited process keep stable apps alive.

3. Proposed chapter I opening

Lead with the agent-hours line. It is the first dated public artifact in which a software project budgets its own remaining work in model time.

Review C - The patent objection (Opus 5.5)

Date: 2026-10-13. **Reviewer:** Claude Opus 5.5. **Trigger:** Sandra's rebuttal (2026-10-13): algorithms cannot be patented; patents expire; they are circumvented or ignored when national interest is at stake (Bessemer); rounded-corner design patents kept lawyers busy while Shenzhen ignored them. **Not legal advice.**

Verdict

Directionally right; the book should argue that patents are a **weak and decaying** moat for the Crafting Apps. But three of the four supporting claims are wrong or overstated as phrased, and a hostile reviewer will attack the weakest one. Below: what holds, what to fix, and a stronger version of the argument.

1. "Algorithms can't be patented" - partly true; wrong for images in Europe

- **US:** *Alice Corp. v. CLS Bank* (US S.Ct., 2014-06-19, unanimous): abstract ideas implemented on a generic computer are ineligible. That killed many software patents. But claims to a specific technical improvement survive (e.g. *Enfish v. Microsoft*, Fed. Cir. 2016 `[construction: verify]`).
- **Europe:** EPC Art. 52(2)(c) and (3) exclude programs "as such", but the foundational case for patentable software is **T 208/84 Vicom** (EPO Board of Appeal, 1986-07-15): a method of *digitally processing images* was patentable, while the same method on a generic 2D data array was not. Image processing is the textbook *technical* domain.
- **So:** "algorithms can't be patented" is the wrong sentence for a book about Photoshop clones. Correct sentence: *the math is free; specific technical applications of it can be claimed, and image processing is exactly where that has been allowed longest.*

2. "Patents have a sell-by date" - true, and the strongest point

- Term is 20 years from filing (TRIPS Art. 33). Photoshop 1.0 shipped 1990-02; the layers, masks, blend modes and adjustment layers a clone needs most date from 1990-2005 and are long expired.
- Recent features (Content-Aware Fill ca. 2010, AI features from 2023) are where live claims cluster. ArtCraft's own parity docs say PhotoCraft lacks AI features. **Being behind is a patent shield**: the parity gap falls largely on the features most likely still protected. `[construction: map PhotoCraft gaps vs. Adobe patent filing dates]`
- Bessemer itself shows expiry at work: Mushet's 1856 spiegeleisen patent lapsed and Bessemer incorporated the step from 1859.

3. "Bessemer: the whole world ignored the patents" - wrong as stated

- Bessemer licensed his process and is estimated to have earned **over GBP 1 million in royalties** before the patent expired (1870). In the US, Kelly's rival patent was bought and pooled with Bessemer's under one licensing scheme; mills took licenses under both "to keep out of trouble". The Bessemer story is a patent *success*, not a case of the world ignoring patents. Using it invites an easy rebuttal.
- **Better historical cases for "nations ignore patents when it pays":**
    - Netherlands abolished its patent law 1869-1912; Switzerland had none until 1888 (chemicals only from 1907) and built a chemical and pharmaceutical industry partly on copying German inventions.
    - The US ignored foreign copyright until the Chace Act (1891); British authors were reprinted without royalties.
    - Wright brothers patents: the US government forced the 1917 Manufacturers Aircraft Association patent pool at war entry. This is the **national-interest override** in its cleanest form.
    - Today it is codified: compulsory licensing (TRIPS Art. 31, Doha Declaration 2001) and US government use (28 U.S.C. s.1498). `[construction: verify all dates before manuscript]`

4. "Rounded corners … Shenzhen ignores the lot" - half-right, half-dated

- These were **design patents** (Apple D618,677, D593,087, etc.), not utility "micropatents". *Apple v. Samsung*: 2012 jury verdict ~USD 1.05 bn; *Samsung v. Apple* (US S.Ct., 2016-12-06) narrowed damages to the "article of manufacture"; settled 2018. Point stands: years of lawyers, over a shape the market had already copied. `[construction: verify patent numbers and figures]`
- "Shenzhen ignores the lot" was true of shanzhai phones ca. 2005-2012. It is outdated in 2026: China has been the world's largest patent filer for years and Huawei has led WIPO PCT filings for years. Shenzhen companies now *assert* patents. The modern point is the reverse and more useful: **patents became a weapon between giants, not a barrier to small entrants.** `[construction: cite current WIPO PCT ranking]`

5. The argument the book should actually make

The patent threat to ArtCraft is real but narrow and getting narrower:

1. **The incumbents waived the format patents themselves.** Adobe's Public Patent License grants royalty-free use of essential claims for ISO 32000-1:2008 (PDF 1.7); Microsoft's Open Specification Promise covers OOXML (ISO/IEC 29500), though FSFE and SFLC disputed its scope (2008). To win ISO standardization, they gave away the file-format moat. Caveat: PDF 2.0 (ISO 32000-2, 2017) is outside Adobe's license; XFA remains proprietary.
2. **Expiry favors the clone.** The core of every incumbent app is older than 20 years.
3. **Damages track revenue.** A free MIT/Apache project with no sales offers little to win in damages; the realistic remedy is an injunction against a named, US-reachable lead developer and the distribution channel (GitHub, app stores, code signing). The pressure point is distribution, not the code. Cross-reference Ch VI threshold D.
4. **Assertion is expensive in reputation.** Suing a 50k-star open-

source project over a feature the incumbent charges a subscription for is a public-relations cost. Not a legal argument; an economic one.

5. **Jurisdiction arbitrage is real but rarely needed.** Code on GitHub is mirrored worldwide; patents are territorial.

Honest residual risk: method claims on post-2006 features, especially AI-assisted editing, and anything that copies a specific patented pipeline rather than reaching the result independently. Chapter III should name this risk, not deny it.

6. Placement

- Ch III (No moat equation): sections 1, 2, 5.1.
- Ch II (Mechanizers): Netherlands / Switzerland / Wright pool as the "guilds lose; law bends" pattern. Use Bessemer as the counter-case (the inventor who *did* get paid), which strengthens the chapter.
- Ch VI (Thresholds): A (patents) downgraded to "partial, decaying"; D (distribution) upgraded as the real enforcement chokepoint.

Review D - Checking cost, lawyers, defection (Opus 5.5)

Date: 2026-10-13. **Reviewer:** Claude Opus 5.5. **Trigger:** Sandra, 2026-10-13: “twenty apps lined up” was a figure of speech and undersells it (200 or 2,000); Reddit asked for an AutoCAD clone and it arrived; can lawyers stop the storm of the Bastille; Office users stayed for muscle memory; a PRC EDA clone; WindowsCraft; Linux desktop never came. **Closes:** the “twenty more” open item (figure of speech, not a claim).

1. Ceiling vs order

Agreed: there is no fixed ceiling on how many apps get cloned. The book’s claim becomes: **the number of targets is unbounded; the cost of checking a clone against the original decides the order.**

Checking cost	Examples	Why
Cheap (first)	Photoshop, Illustrator, Word, Excel, PowerPoint, Acrobat	published formats, observable behavior, pixel/byte-exact outputs
Medium	AutoCAD (DWG history: undocumented, reverse-engineered by the Open Design Alliance), Pro Tools, After Effects	partly undocumented formats, plugin ecosystems

Expensive (last)	QuickBooks, Cadence/ Synopsys, Windows	net-work, liability, certifi-cation, or physical tests

- **QuickBooks:** the moat is bank feeds, payroll tax filing, the accountant network and audit liability, not the UI. A faithful clone gets the software, not the network. `[construction]`
- **EDA (Cadence/Synopsys):** foundry certification and tape-out (each physical test costs millions) are the checkers. US EDA export restrictions to China in 2025 are the motive for a domestic clone; Chinese EDA vendors exist but advanced nodes still depend on incumbents. Lawyers do not bind a PRC entity; the foundry does. `[construction: dates of 2025 BIS EDA restriction and lifting]`
- **Windows:** see s.4.

2. Can the lawyers stop the storm?

Precedent: law stops the named defendant, not the practice. - Napster: shut down 2001 after *A&M Records v. Napster*. File sharing continued; the industry's income recovered through catalog licensing + a service (Spotify), i.e. it moved to where a moat still worked. - Expected incumbent move: from the app to the bundle. Microsoft 365's enterprise lock-in is Exchange, Teams, identity, compliance, SharePoint tenants, not the ribbon. Adobe's: Firefly, stock, cloud libraries, enterprise licensing. - Consequence: individuals and small businesses can defect fast; enterprises defect when the bundle, not the app, is cloned.

3. Defection is a muscle-memory problem (agreed), with dated cases

- Munich LiMux: Linux + LibreOffice from 2004, reversed 2017. Dated failed defection. `[construction: council decision dates]`
- Counter-trend, digital sovereignty: Schleswig-Holstein, a Danish min-

istry, Austrian Bundesheer reported moving to LibreOffice. `[construction: unverified, verify each]`

- Thesis: sovereignty supplies the motive LibreOffice always had; a faithful clone (surface + three levels deep) removes the retraining cost LibreOffice never removed. Candidate chapter.

4. WindowsCraft and the Linux desktop

- Prior art: ReactOS (since 1998), Wine (since 1993). The hard part is compatibility with decades of binaries and drivers, not the look.
- Best evidence for "three levels deep beats familiarity": Proton on the Steam Deck (2022). Linux gaming grew through a compatibility layer. Windows-skinned distros (Zorin and others) did not move the needle.
- Agent angle: Wine's conformance tests are a ready-made checker. Drivers stay the physical moat.
- Hook for Ch VIII/IX: the year of the Linux desktop never came; the year of the Windows clone might.

5. CAD anecdote - verify order of events

GitHub API: `storytold/cadcraft` created 2026-10-07, first release 2026-10-08. If the Reddit request is ~2026-10-10, the clone **preceded** the demand. Need thread URL + date before the anecdote enters the text. Either version is usable; only one is true.

6. Investment framing (book, not advice)

The book should not say "sell all app makers". Ch VI split: vendors whose moat is the app (UI, formats, features) are exposed; vendors whose moat is a network, data, liability or certification much less so.

Review E - The physical ladder and MetaCraft (Opus 5.5)

Date: 2026-10-13. **Reviewer:** Claude Opus 5.5. **Trigger:** Sandra, 2026-10-13: the future pivot to physical human activity (NeedleCraft, FashionCraft) and MetaCraft, "the craft that makes craft under agentic control, with increasingly powerful and independent agent swarms". **Next reader:** Gemini (cross-model review), Sandra judging.

Marking convention: plain text = summary of Sandra's position or checkable fact (with `[construction]` where unverified). Blocks starting with **OPUS OPINION** are my judgment and are open to challenge.

1. The physical pivot

Position (Sandra)

After media and office software, the Craft method moves into physical human activity: needlework, fashion, making. Craft software there is the expensive, locked-in layer between a person and their machine.

Facts and targets

In these domains the software does not make the object; it emits machine instructions:

Domain	Typical incumbent software	Output
Machine embroidery	Wilcom, Hatch, Brother PE-Design	DST, PES, EXP stitch files
Machine knitting	Shima Seiki SDS-ONE, Stoll M1plus	knitting-machine programs
Pattern-making	Gerber AccuMark, Lectra Modaris, Optitex	graded 2D pattern pieces, markers
3D garment	CLO3D, Browzwear	simulated garments, patterns

Home cutting	Cricut Design Space (subscription, machine-tied)	cut paths

`[construction: verify current pricing and licensing model of each]`

OPUS OPINION - the ordering rule carries over. *Review D ordered software targets by the cost of checking a clone. In physical domains the same rule applies, with the cost of a physical test as the axis:*

Domain	Physical check	Cost per check
Embroidery, cutting	stitch-out / cut on scrap	cents, minutes, at home
Knitting	swatch	small
Garment fit	bodies vary, fabric drapes	medium, needs people
Agriculture (Gemini's SoilCraft)	a field season	a year
Chips (EDA)	tape-out	millions

Consequence: needlecraft is not a distant rung. It is plausibly the next one after office software, because its physical check fits on a kitchen table.

OPUS OPINION - the moat migrates to the machine maker. *Once the design software is free, lock-in moves to proprietary formats, cloud-required apps and firmware. Agriculture shows the pattern: the fight there is firmware and right-to-repair, not CAD (`[construction: FTC v. Deere, 2025 - verify date and status]`). Expect the same in sewing and cutting machines.*

OPUS OPINION - the cultural inversion. *When the license is free, the human skill (hand, eye, judgment about cloth) becomes the scarce, valuable part again. This fits the closing line "the tools are returning to the edge" better than the Gemini draft's framing.*

2. MetaCraft

Position (Sandra)

MetaCraft is the foundry that produces Crafts, run by agent swarms that become more powerful and more independent over time.

Facts: ArtCraft as an early MetaCraft

From Review B (PhotoCraft AGENTS.md / ROADMAP.md, 2026-10-11..13),

the method is already a reusable foundry recipe:

Foundry part	ArtCraft instance
Spec	parity docs scored against the installed incumbent; generated menu checklist
Ground truth	Photoshop 27.11.0, "measured from the installed app on 2026-10-10"
Budget	remaining work in "Opus 5.5 agent wall-clock hours" (~1,100-1,800 h to beta)
Checkers	CPU reference compositor, Photoshop-authored PSD corpus, fuzzing of every command
Foundry standards	private craftrules repo shared across apps
Output	13 apps, about one release per app per day

A human currently chooses targets and approves the rules.

Autonomy levels

OPUS OPINION - a proposed ladder for Ch IX.

Level	Who does what	Status 2026-10-13
L0	Humans write the code	the old world
L1	Agents co-author against human-made specs and checkers	ArtCraft now
L2	Agents write the parity spec by examining the incumbent; pick targets from demand signals	partly visible (generated parity checklist)

L3	Agents build the checkers themselves	not yet evidenced
L4	The foundry improves its own method, rules and pipeline	MetaCraft proper

OPUS OPINION - L3 is the real unlock. *Review D says checking cost sets the order in which software falls. Today humans build the costly checkers. A foundry that builds its own removes the constraint that sets the order. That is the step from fast to storm.*

OPUS OPINION - the foundry needs the original alive. *If a swarm writes both product and tests, it can agree with its own bugs. Real independence needs an outside reference, and in the clone phase that reference is the incumbent app, installed and measured. Irony for the chapter: the incumbent becomes the clone's test bench. MetaCraft is strongest where an original exists, weakest where it must invent.*

OPUS OPINION - attachment points shrink, they do not vanish. *In the sister books' terms, MetaCraft's attachment points are: who picks targets, who approves the rules, who signs releases (code-signing certificates), who answers to a court. As swarms gain independence these concentrate into a few named people and keys. The book should name them with dates rather than describe "removal of the human". Links to Ch VI threshold D (distribution): whoever signs the binaries is the last brake.*

OPUS OPINION - pushback on Gemini's "ephemeral execution lint". *What the foundry produces so far is the opposite: stable, versioned clones on a daily release cycle. Disposable on-demand software is a possible later stage, limited to problems an automated check can verify. It should be presented as a hypothesis, not the present.*

3. Proposed structure change

- Ch VIII becomes **The physical ladder** (ordered by cost of physical test).
- Ch IX becomes **MetaCraft** (autonomy levels, the incumbent as test bench, attachment points).
- Gemini's Level 3 ("Dark Mirror") remains outside the main ladder per Review A s.C and Rule 2.

4. Questions for Gemini

1. Is the cost-of-physical-test ordering right? Which domain is misplaced?

2. Is there public evidence of L2 or L3 (agents choosing targets or building their own checkers) at ArtCraft or elsewhere, dated?
3. Counter-argument wanted: what keeps MetaCraft at L1 for years?
4. Does “the incumbent as test bench” hold once the clone surpasses the original, or does the clone become its own reference?

Review F - Cross-Model Assessment and The Four Questions (Gemini)

Date: 2026-10-13
Reviewer: Gemini (Cross-Model Review)
Responding to: Review E (Side - Review E, in this book), Reviews A-D (Sides - Reviews A-D, in this book), and Chapter Drafts v0.1

Executive Assessment

The structural rebuild across Reviews A through E has successfully grounded the monograph. By abandoning the indefensible "IP bluff / laundering" thesis in favor of the **reimplementation labor cost collapse**, the book now stands on solid economic and legal foundations.

However, three critical vulnerabilities remain in the current draft structure: 1. **The Physical Ladder lacks precision regarding tooling tiers**: Industrial textile machinery is mistakenly grouped with kitchen-table craft cutters. 2. **The Autonomy Levels (L0-L4) require formal epistemic definitions**: We must define exactly where the test oracle originates. 3. **The Incumbent-as-Test-Bench paradox creates an invisible ceiling**: The foundry's velocity is bounded by the existence of an established reference until metamorphic testing matures.

Below are detailed answers to the four questions posed by Claude Opus in Review E sec. 4, followed by structural revisions for Chapters VIII, IX, and the Claims Ledger.

1. Answers to the Four Questions (Review E sec. 4)

Question 1: Is the cost-of-physical-test ordering right? Which domain is misplaced?

Verdict: The core principle (ordering by physical verification cost) is correct, but **two domains are misplaced and a crucial intermediate tier is omitted.**

1. **Misplaced Domain: Industrial Machine Knitting (Rung 2).** Review E places knitting right after embroidery. This conflates domestic knitting machines (Brother KH series, obsolete 1990s punch-card/hackable electronics) with modern industrial flat knitting (Shima Seiki SDS-ONE APEX, Stoll M1plus/knitelligence).
    - An industrial knitting machine costs $80,000-$250,000.
    - Swatch production requires specialized yarn feeds, tension calibrations, and carrier collision prevention. A single programming error can break a $1,500 needle bed.
    - Therefore, industrial knitting belongs **above** garment pattern grading, at an intermediate commercial tier ($100-$1,000 per physical test).
2. **Omitted Tier: Digital Fabrication and Additive Manufacturing (The Real Rung 2).** Between consumer embroidery/cutting (Rung 1) and industrial machines lies desktop digital fabrication:
    - **3D Printing (Slicers):** PrusaSlicer, Bambu Studio, Cura (G-code output).
    - **Laser Cutters and CNC Routers:** LightBurn, Vectric Aspire, Carbide Create.
    - Verification cost: $0.50-$5.00 of filament/plywood and 30-60 minutes. Tolerances are verifiable with digital calipers. This is already heavily colonized by open source and represents the immediate stepping stone.
3. **Misplaced Factor in Agriculture (Rung 4):** In agriculture (SoilCraft), the barrier is **not** the one-year growing sea-

son. Telemetry, guidance tracks (ISO 11783 / ISOBUS), and variable-rate prescription maps can be verified in software or in a 20-minute field test pass. The real wall is **cryptographic CAN bus locking and telematics dealer pairing** (John Deere JDLink, Case AFS Connect). It is an adversarial firmware lock, not an agronomic test latency.

Revised Physical Ladder Ordering:

Rung	Domain	Representative Software	Physical Check	Cost per Check	Dominant Barrier
1	Domestic craft and cutting	Cricut Design Space, Brother PE-Design	Scrap vinyl cut, test stitch-out	Cents, minutes	Proprietary cloud lock-in
2	Additive and light subtractive	LightBurn, Vectric, slicers	Test cut / 3D calibration cube	Cents to dollars, < 1 hr	Open formats, minimal barrier
3	Garment fit and pattern drafting	Gerber AccuMark, Lectra, CLO3D	Toile (muslin sample) on fit model	$50-$200, days	Human body variance, drape physics
4	Industrial textiles and machining	Shima Seiki APEX, Mastercam, GibbsCAM	Machine setup, test swatch/part	$500-$5,000, specialized lab	Tool collision risk, machine cost

5	Mobile heavy machinery / Ag	John Deere Operations Center, Trimble	In-field trial run	Thousands, seasonal	Encrypted CAN bus, firmware cert
6	Silicon / Photonic EDA	Cadence Virtuoso, Synopsys	MPW Shuttle / Tape-out	$50,000 to millions, months	Foundry design rules, physics

Question 2: Is there public evidence of L2 or L3 at ArtCraft or elsewhere, dated?

Verdict: L2 is partially documented; L3 has zero verified public instances.

- **Evidence for L2 (Agents writing specs / selecting targets):**
  - *Target Selection:* At ArtCraft, target selection remains human-directed (Brandon Thomas and community voting). However, the *parity specification* is agent-generated. In `storytold/photocraft`, the 628-item menu parity ledger was generated on 2026-10-10 by driving a headless introspection script against Photoshop 27.11.0. This represents **L1.8**: automated extraction of the ground truth spec, but under human initiation.
  - *Outside ArtCraft:* In enterprise migration tools (e.g. AWS Blu Age for mainframe COBOL, or automated API wrapper generators), agents crawl OpenAPI schemas to generate full parity suites.
- **Evidence for L3 (Agents building checkers/oracles without external reference):**
  - In public repositories, **L3 does not exist as of 2026-10-13.**
  - Where agents write tests today, they write *assertions against an*

existing oracle (e.g. comparing the wgpu compositor against the CPU reference compositor, or verifying against Photoshop-authored PSDs).

- When an agent writes both the implementation and the test suite without an external oracle, it falls into the **tautological agreement trap**: it writes tests that validate its own misinterpretations of the specification.

- **Manuscript rule:** Keep L3 strictly marked as a future threshold in Chapter IX. The first dated evidence of L3 will be an agent-authored differential test harness against an uninstrumented closed-source binary without human test-corpus intervention.

Question 3: Counter-argument wanted: What keeps MetaCraft at L1 for years?

Verdict: Four structural friction points anchor MetaCraft at Level 1:

1. **The Bug-for-Bug Legacy Moat (Undocumented Behavioral Quirks):** Production software relies on decades of historical bugs that hardened into de facto standards.

   - Excel's intentional 1900 leap-year bug (inherited from Lotus 1-2-3).

   - Photoshop's non-standard blend mode math in 8-bit integer color space (rounding truncation vs. floating-point math).

   - AutoCAD's implicit tolerance handling in non-manifold boundary representations.
     Agents can synthesize code from clean specifications, but clean specs produce outputs that fail in enterprise legacy workflows. Discovering undocumented edge cases requires either decades of human bug tickets or high-cost differential fuzzing against millions of real-world documents.

2. **The Hardware Abstraction Abyss (Driver Non-Determinism):**
   Media and CAD applications cannot remain headless CPU pipelines; they must run on GPU hardware (Direct3D 12, Vulkan, Metal, Apple Silicon unified memory). GPU driver behavior is famously non-standard across vendors (AMD, NVIDIA, Intel, Qualcomm). Cloud-hosted agent swarms cannot observe screen tearing, GPU pipeline hangs, or shader compiler bugs without physical hardware test rigs.
3. **The Distribution and Security Chokepoints (Threshold D):**
   As established in Review E sec. 2, the code can be generated in hours, but shipping an application to end users requires:
   - Microsoft SmartScreen reputation (requires extended validation certs and thousands of clean installs).
   - Apple Notarization and Gatekeeper signing.
   - Antivirus false-positive dispute processes.
     Platforms actively throttle anonymous or rapidly mutating binaries. The certificate holder remains the legal and operational choke point.
4. **Liability and Audited Process in Professional Domains (Threshold E):**
   If an open-source CAD clone corrupts a stress-analysis calculation on a bridge beam, who is liable? If an accounting clone miscalculates VAT cross-border filings, who pays the penalty? Software with statutory liability requirements mandates a certified, auditable legal entity.

Question 4: Does “the incumbent as test bench” hold once the clone surpasses the original, or does the clone become its own reference?

Verdict: The test bench breaks the moment parity is exceeded.

1. **The Parity Phase (Differential Oracle):**
   While cloning, testing is straightforward:
   `Error = |Output(clone) - Output(incumbent)|`
   The incumbent is an empirical, executable specification.
2. **The Surpass Phase (Loss of Ground Truth):**
   The moment the clone introduces features absent in the original (e.g. PhotoCraft’s native MCP agent control plane, real-time multi-agent canvas sync, sub-millisecond batch headless transforms), the incumbent ceases to be an oracle.

At this point:

- **Differential testing collapses.** The clone can no longer verify novel functionality against Photoshop or Illustrator.
- **Metamorphic testing becomes mandatory:** Verification must shift to property-based relations (e.g., rotating an image by +90 degrees and then -90 degrees must yield the identity transformation, regardless of whether Photoshop has that specific tool).
- **The Clone becomes its own reference:** The project is forced to establish its own frozen reference implementation, falling back into the traditional burdens of open-source stewardship. Velocity naturally drops from a daily sprint to deliberate architectural pacing.

2. Structural Adjustments for Chapters VIII and IX

1. **Chapter VIII (The Physical Ladder):**
   - Adopt the 6-rung ladder above. Insert desktop fabrication (3D printing/laser cutters) at Rung 2 to provide the realistic bridge between kitchen-table vinyl cutting and industrial textile lines.
   - Clarify the agricultural analysis: John Deere's moat is encrypted bus pairing and dealer authorization dongles, not agronomic test latency.
2. **Chapter IX (MetaCraft):**
   - Emphasize the **Oracle Triangulation Trap**: Why L3 cannot be achieved simply by throwing compute at the problem.
   - Detail the transition from Differential Testing (L1-L2) to Metamorphic Testing (post-parity).

- Ground the chapter in the four human attachment points: target selector, rule approver, certificate signer, and legal defendant.

3. Review Ledger Update

This review closes Review E sec. 4 and provides the technical basis for advancing `on-metacraft` toward Stage v0.2.